

A
Vocational Training Report
On

NexusPrep: A Smart Study Assistance Platform

Submitted to



BHILAI INSTITUTE OF TECHNOLOGY, DURG
Artificial Intelligence and Web Development

Affiliated to

**CHHATTISGARH SWAMI VIVEKANAND
TECHNICAL UNIVERSITY, BHILAI**

In partial fulfilment of Vocational Training

In

Artificial Intelligence and Web Development

By

ANUPAM SHARMA, 6TH SEM, 300102223023

Under the Guidance of

Himanshu Chaudhary

(Industrial In-charge)

CERTIFICATE OF COMPLETION

DECLARATION

I hereby declare that the vocational training report entitled “**NexusPrep: A Smart Study Assistance Platform**” is based on my own work carried out during the training period at **Aimaxa Tech, Bhilai**, under the guidance of **Mr. Himanshu Chaudhary**.

I assert the statements made and conclusions drawn are an outcome of my research and training work. The work contained in the report is original and has been done by me under the general supervision of my supervisor. Also, the work has not been submitted to any other Institution for any other degree/diploma/certificate in this university or any other University of India or abroad. I have followed the guidelines provided by the university in writing the report. Whenever I have used materials (data, theoretical analysis, and text) from other sources, I have given due credit to them in the text of the report and giving their details in the references.

(Signature of the Candidate)

Name: ANUPAM SHARMA

Roll No: 300102223023

Enrollment No: CD8391

ACKNOWLEDGEMENT

I take this opportunity to extend my sincere gratitude to **Aimaxa Tech, Bhilai**, for providing me with an incredible platform to gain hands-on experience in the cutting-edge fields of Generative AI, Large Language Models (LLMs), and modern Microservices Architecture. This training was a significant step in bridging the gap between theoretical knowledge and practical software application.

First and foremost, I would like to thank **Mr. Himanshu Chaudhary**, our Course Coordinator, for his unwavering guidance, constant motivation, and invaluable insights throughout the training period. His expertise and support were instrumental in the successful completion of this project.

I am also deeply grateful to the faculty and staff of **Aimaxa Tech** for creating a stimulating learning environment, providing the necessary resources, and for their timely assistance whenever needed.

Last but not the least, I express my heartfelt thanks to **Dr. Sunita Soni (HOD)**, Department of Computer Science and Engineering, Bhilai Institute of Technology, Durg, and all my professors for their encouragement and for giving me the opportunity to undertake this enriching vocational training.

Name: ANUPAM SHARMA

Roll No: 300102223023

Enrollment No: CD8391

ABSTRACT

In today's highly competitive and fast-paced educational environment, effective study strategies are critical factors influencing academic performance, cognitive load management, and long-term knowledge retention. Traditional study approaches often rely on passive reading or manual note-taking, which fail to combat the psychological phenomenon of memory decay and lack the dynamic adaptability required for targeted exam preparation. To address these challenges, this project presents NexusPrep, an Intelligent Academic Strategist powered by advanced Artificial Intelligence (AI) and cognitive science methodologies.

The system leverages raw academic materials—such as textbooks, lecture notes, and Previous Year Question Papers (PYQs)—to generate optimized, personalized study workflows. Using a decoupled microservices architecture (React.js and FastAPI) integrated with Google's Gemini AI via Direct Context-Aware Retrieval, the system reliably extracts complex academic text and strict, double-escaped LaTeX mathematics. A core innovation of the platform is its semantic validation gatekeeper, which cross-references syllabus notes with PYQs to predict high-yield topics and generate highly probable descriptive exam questions.

The proposed NexusPrep ecosystem offers a scalable, data-driven, and cloud-native solution for students and educators to adopt advanced, highly strategic study practices traditionally reserved for premium academic coaching. The report also discusses future directions, including the integration of custom-trained embedding models for localized vector search, expansion into collaborative multi-user study lobbies, and the integration of open-source codebase APIs for software engineering students.

This project demonstrates the feasibility and effectiveness of combining generative AI analytics with established cognitive psychology, ultimately bridging the gap between archaic rote memorization models and intelligent, automated educational solutions for the future of learning.

TABLE OF CONTENTS

S.No.	Section / Chapter	Page No.
1	Cover Page	i
2	Certificate of Completion	ii
3	Declaration	iii
4	Acknowledgement	iv
5	Abstract	v
6	Table of Contents	vi
7	List of Figures	ix
8	List of Tables	x
	CHAPTER 1: INTRODUCTION	1-8
1.1	Background and Context	1
1.2	Motivation for the Project	2
1.3	About the Organization and the Vocational Training Program	3
1.4	Problem Statement	4
1.5	Introduction to NexusPrep	5
1.6	Objectives of the Project	7
1.7	Scope of the Project	8
	CHAPTER 2: LITERATURE REVIEW	9-20
2.1	Introduction to the Literature Review	9
2.2	The Forgetting Curve and the Psychology of Memory	9
2.3	Spaced Repetition Systems in Practice	11
2.4	Personalization and Learner Context in Educational Technology	12
2.5	Document Parsing, OCR, and the Text Extraction Problem	12
2.6	Automated Text Summarization	13
2.7	Automated Question Generation	14
2.8	Large Language Models and the Gemini API	15
2.9	Comparative Review of Existing Study and Ed-Tech Platforms	16
2.10	Literature on the Supporting Technology Stack	18
2.11	Summary and Positioning of the Present Work	19
	CHAPTER 3: METHODOLOGY	21-40
3.1	Overview of the Development Approach	21
3.2	System Architecture	21

S.No.	Section / Chapter	Page No.
3.3	Technology Stack: Choices and Justification	23
3.4	Database Design	25
3.5	Document Ingestion and the OCR Fallback Pipeline	27
3.6	AI Orchestration and Prompt Engineering	29
3.7	Mathematical Notation Rendering Pipeline	32
3.8	The Spaced Repetition Engine	33
3.9	Authentication, Security, and Session Handling	34
3.10	Frontend Design and User Flow	35
3.11	API Design Summary	39
3.12	Testing and Validation Approach	40
3.13	Summary of the Methodology	40
	CHAPTER 4: RESULTS AND DISCUSSION	41-50
4.1	Overview of the Evaluation Approach	41
4.2	Functional Results: Normal Mode	41
4.3	Functional Results: Exam Mode	43
4.4	Reliability and Fallback Behavior in Practice	44
4.5	The Spaced Repetition System in Practice	45
4.6	Frontend and User Experience Outcomes	46
4.7	Informal Performance Observations	47
4.8	Mapping Results Back to the Project Objectives	47
4.9	Discussion: Strengths, Trade-offs, and Honest Limitations	49
4.10	Summary	50
	CHAPTER 5: FUTURE SCOPE	51-56
5.1	Introduction	51
5.2	Toward an Adaptive Spaced-Repetition Engine	51
5.3	Strengthening Security and Account Management	52
5.4	Improving Document Ingestion and Giving Users Control Over It	52
5.5	Building a Genuine Automated Test Suite	53
5.6	Deeper Personalization	53
5.7	Integration with Existing Academic and Institutional Systems	54
5.8	Mobile Access and Offline Support	54
5.9	Scalability and Infrastructure Considerations	55
5.10	Ethical Considerations for Future Development	55

S.No.	Section / Chapter	Page No.
5.11	Summary	56
	CHAPTER 6: CONCLUSION	57-59
6.1	Summary of the Project	57
6.2	Reflections on the Vocational Training Experience	58
6.3	Personal and Academic Growth	58
6.4	Closing Remarks	59
	REFERENCES	60
	BIBLIOGRAPHY	61

LIST OF FIGURES

Fig. No.	Description	Page No.
1.1	NexusPrep Homepage	6
2.1	The Ebbinghaus forgetting curve	10
3.1	NexusPrep Architecture Diagram	22
3.2.1	Import block for App.jsx libraries	24
3.2.2	Import block for main.py libraries	25
3.3	NexusPrep entity-relationship diagram	26
3.4	PDF processing flow chart	28
3.5	Exam Mode request lifecycle flowchart	31
3.6	Ebbinghaus curve with NexusPrep's five scheduled review points	34
3.7.1	Normal Mode Results dashboard with Key Takeaways, View Documents, and the flashcard/quiz launch option	36
3.7.2	Normal Mode Results dashboard with generated Key Formulas	36
3.8.1	Flashcard revision interface with an un-flipped card showing card title	37
3.8.2	Flashcard revision interface with a flipped card showing definition	37
3.9.1	Exam mode results dashboard showing yield topics and strategic insights	37
3.9.2	Exam mode results dashboard showing predicted questions	38
3.9.3	Split Screen interface to access course material	38
4.1	ResultsDashboardView after processing a Normal Mode document	42
4.2	Exam Mode Results dashboard	44
4.3	QuizView interface with multiple-choice questions and a timer.	44
4.4	Saved Decks library in ProfileView showing saved flashcards with next revision date	46

LIST OF TABLES

Table No.	Description	Page No.
2.1	Comparative Overview of Existing Study Platforms vs. NexusPrep	18
3.1	NexusPrep API Endpoint Reference	39
4.1	Project Objectives vs. Achieved Outcomes	48

CHAPTER 1

INTRODUCTION

1.1 Background and Context

Education, as a field, has always been slower than most other sectors to absorb technological change. For decades, the basic mechanics of learning — reading a textbook, attending a lecture, writing notes by hand, revising before an exam — remained largely untouched even as computing power reshaped nearly every other part of daily life. That resistance to change has, however, started to crack in the last several years. The widespread availability of smartphones, the drop in the cost of cloud computing, and, more recently, the arrival of large language models capable of understanding and generating natural language, have together opened a door that education technology companies have been eager to walk through.

What makes this moment particularly interesting is not simply that computers can now "understand" text better than before, but that they can do so cheaply enough and quickly enough to be embedded directly into a student's daily study routine. A decade ago, building a system that could read a scanned textbook chapter, extract the core ideas, and turn them into a structured revision aid would have required a small army of engineers, a custom-trained model, and a budget well outside the reach of a student project or a small startup. Today, the same outcome is achievable by combining a handful of well-documented APIs, a reasonably designed backend, and a frontend that knows how to present information without overwhelming the user. This shift in accessibility is the backdrop against which the present project, NexusPrep, was conceived and built.

At the same time, the sheer volume of study material a student is expected to process has not gone down — if anything, it has increased. Competitive examinations, university coursework, and professional certifications all demand that a learner absorb large quantities of information under real time pressure, and then be able to recall that information reliably, weeks or months after first encountering it. Traditional study habits — highlighting a textbook, rewriting notes, cramming the night before an exam — are demonstrably inefficient when judged against what cognitive science actually tells us about memory formation and retention. Concepts like spaced repetition and active recall have existed in academic literature for a long time, yet very few students actually apply them in a disciplined way, largely because doing so manually is tedious and easy to abandon.

This is the specific gap that motivated the vocational training project undertaken during this internship: not the invention of a new learning theory, but the practical, engineering-driven task of making an already well-understood theory (spaced repetition, in particular the Ebbinghaus forgetting curve) usable by an ordinary student without requiring them to manage spreadsheets, timers, or physical flashcards. Layered on top of that is a second, equally practical problem — most students do not have neatly organized notes to begin with. Their material exists as scanned PDFs, photographed textbook pages, messy lecture slides, and previous years' question papers collected from seniors. Any tool that expects "clean" input before it can be useful is, in practice, a tool most students will never use.

1.2 Motivation for the Project

The motivation for this project came from a fairly ordinary observation, one that most students and educators have made at some point: the effort required to prepare good study material is often just as large as the effort required to actually learn from it. A student who wants a set of flashcards for a chapter first has to read the chapter, decide what is important, phrase each concept as a question-and-answer pair, and then repeat this process for every chapter in the syllabus. By the time this preparation work is done, a meaningful chunk of the time originally available for revision has already been spent.

Artificial Intelligence, and specifically the recent generation of large language models such as Google's Gemini family, changes this equation. These models are capable of reading a document, understanding its structure, and generating structured educational content from it — summaries, key formulas, flashcards, and even likely exam questions — in a matter of seconds. The technology to do this is not new in the sense of being unavailable; what is new is that it has become affordable, fast, and reliable enough to be placed directly in front of an end user without requiring that user to have any technical background whatsoever.

A second and equally important motivation came from noticing how differently students prepare depending on whether they are studying for continuous coursework or for a specific, high-stakes examination. When preparing for regular coursework, a student mostly needs comprehension: a clear explanation of the material, the important formulas, and a way to test their own recall. When preparing for a competitive or semester-end examination, the priorities shift — the student wants to know which topics are asked most often, which questions carry more marks, and where the previous years' question papers overlap with the current syllabus. Very few existing tools treat these as genuinely different modes of studying; most simply offer

a single, generic "summarize this document" feature and leave the rest to the student. This distinction — between comprehension-oriented studying and examination-strategy-oriented studying — became one of the defining design decisions behind NexusPrep, eventually taking shape as its two operating modes, referred to throughout this report as Normal Mode and Exam Mode.

Finally, there was a purely technical motivation, one more personal to the experience of the vocational training itself. Building a system that combines a Python-based backend, a modern JavaScript frontend, a cloud database, and a third-party AI service is a genuinely full-stack undertaking. It touches authentication and security, asynchronous programming, document parsing, prompt engineering, database schema design, and user interface design, all within a single coherent product. Very few academic exercises offer the opportunity to work across this entire spectrum on one project, which made NexusPrep an unusually complete vehicle for vocational learning, well beyond what a narrower, single-technology assignment could have offered.

1.3 Overview of the Vocational Training Program

The vocational training that forms the basis of this report was undertaken at **Aimaxa Tech**, Bhilai, in the domain of Artificial Intelligence and Web Development, under the guidance of **Mr. Himanshu Chaudhary**, Industrial In-charge. Aimaxa Tech's training approach centers on pairing trainees with a real, end-to-end product build rather than isolated exercises, reflecting a broader industry pattern in which AI capability is rarely valuable on its own — a model that can summarize a document or answer a question only becomes useful once it is wrapped in a secure, usable interface and connected to a persistence layer that remembers a user across sessions, and the training was structured around internalizing that reality.

The training itself followed a structured but flexible progression. The early phase focused on strengthening foundational skills: Python for backend logic, asynchronous programming patterns (necessary since a responsive web backend cannot afford to block on slow operations like AI generation calls or database queries), and the basics of the React framework for building interactive frontends. Alongside this, time was spent understanding how modern AI APIs are actually consumed in production — not as a single function call that magically returns a perfect answer, but as a service with rate limits, occasional failures, and output that needs to be validated and sometimes retried before it can be trusted.

As the training progressed, focus shifted toward the specific architecture NexusPrep would eventually use: a FastAPI backend responsible for all business logic and AI orchestration, a MongoDB database for persistent storage, and a React-based single-page frontend responsible for everything the user actually sees and interacts with. Considerable attention was given to *why* this particular split of responsibilities makes sense — for instance, why AI orchestration and prompt construction belong on the backend rather than the frontend (to protect API keys and centralize error handling), and why a NoSQL, document-oriented database like MongoDB is often a more natural fit than a relational database when the data being stored — flashcard decks, in particular — has a variable, nested structure that does not map cleanly onto fixed table columns.

The training also placed strong emphasis on defensive engineering: designing a system that degrades gracefully rather than failing outright when something goes wrong. This shows up throughout NexusPrep in small but deliberate ways — for example, if a Gemini AI model is temporarily unavailable or rate-limited, the backend automatically retries against a different model in a fallback list rather than immediately failing the user's request; if an email genuinely fails to send, the backend raises a clear error rather than reporting a false success; and if a PDF cannot be read using standard text extraction because it is a scanned image rather than digital text, the system automatically falls back to an OCR-based extraction path instead of rejecting the file outright.

1.4 Problem Statement

Bringing together the observations discussed above, the core problem this project set out to address can be stated fairly directly:

Students working with raw, unstructured study material — textbooks, scanned notes, and previous years' question papers — lack an accessible tool that can convert this material into structured, retention-optimized study aids, while also distinguishing between the different needs of general comprehension-based study and targeted, examination-focused preparation.

This problem statement carries several implicit sub-problems that had to be addressed individually during development:

1. **The input problem.** Study material rarely arrives in a clean, machine-readable format. A tool that only works on neatly typed text is of limited practical use, since a large proportion of real student material is scanned or photographed rather than natively digital.
2. **The extraction and structuring problem.** Even once text has been extracted from a document, deciding what is important — which sentences represent a key concept, which

numbers represent a formula worth remembering, which sentence is simply narrative filler — is a non-trivial task that historically required human judgment.

3. **The retention problem.** Producing study material is only half the challenge; retaining what has been studied over time is a separate, well-documented cognitive challenge. Without a structured revision schedule, most information reviewed once is substantially forgotten within days, following the pattern described by Hermann Ebbinghaus's forgetting curve, discussed further in Chapter 2.
4. **The dual-mode problem.** A student preparing for a specific exam has fundamentally different needs from a student trying to understand a new topic for the first time, yet most existing study tools do not distinguish between the two, offering the same generic feature set regardless of context.
5. **The accessibility and continuity problem.** Any solution built for this purpose needs to be usable without technical expertise, and needs to preserve a student's progress reliably across devices and sessions rather than tying their study history to a single browser's local storage, which can be cleared, lost, or simply unavailable on a different device.

NexusPrep, the platform developed during this vocational training, was designed as a direct response to each of these sub-problems.

1.5 Introduction to NexusPrep

NexusPrep, described in its own documentation as "The Intelligent Academic Strategist," is a full-stack, AI-driven study assistance platform built to convert raw academic material into interactive, structured study experiences. At a high level, the platform accepts documents from the user — textbook chapters, lecture notes, formula sheets, or previous years' question papers — and uses Google's Gemini large language model to extract meaningful academic content from them, which is then presented to the user through a purpose-built web interface.

The platform is built around two distinct operating modes, each addressing a different studying scenario described in the problem statement above:

- **Normal Mode** is intended for general comprehension. A student uploads one or more PDFs of study material, and the system produces a subject title, a set of chapter-wise summary points (referred to as takeaways), a list of extracted formulas rendered with proper mathematical notation, key term definitions, and a deck of fifteen to twenty flashcards automatically generated from the content.

- **Exam Mode** is intended for targeted, examination-focused preparation. Here, the student uploads two distinct sets of documents — a set of study notes and a set of previous years' question papers (PYQs) — and the system first verifies, using AI-based semantic validation, that the two sets of documents actually belong to the same subject, before cross-referencing them against one another. The output in this mode includes high-yield topics (topics that appear repeatedly across past question papers), strategic insights about how a student should prioritize their time, more than ten predicted descriptive questions split by mark-weightage (4-mark and 8-mark style questions), a quiz composed of intentionally challenging multiple-choice questions, and a flashcard deck built from the same underlying material.

Every flashcard deck the system produces, regardless of which mode generated it, is stored persistently on the backend in a MongoDB database and is automatically scheduled for revision using a spaced-repetition algorithm modeled on the Ebbinghaus forgetting curve. This means a student's study progress is never tied to a single device or browser session — logging in from a different laptop or phone brings back the exact same set of decks, along with an accurate record of when each deck is next due for revision.

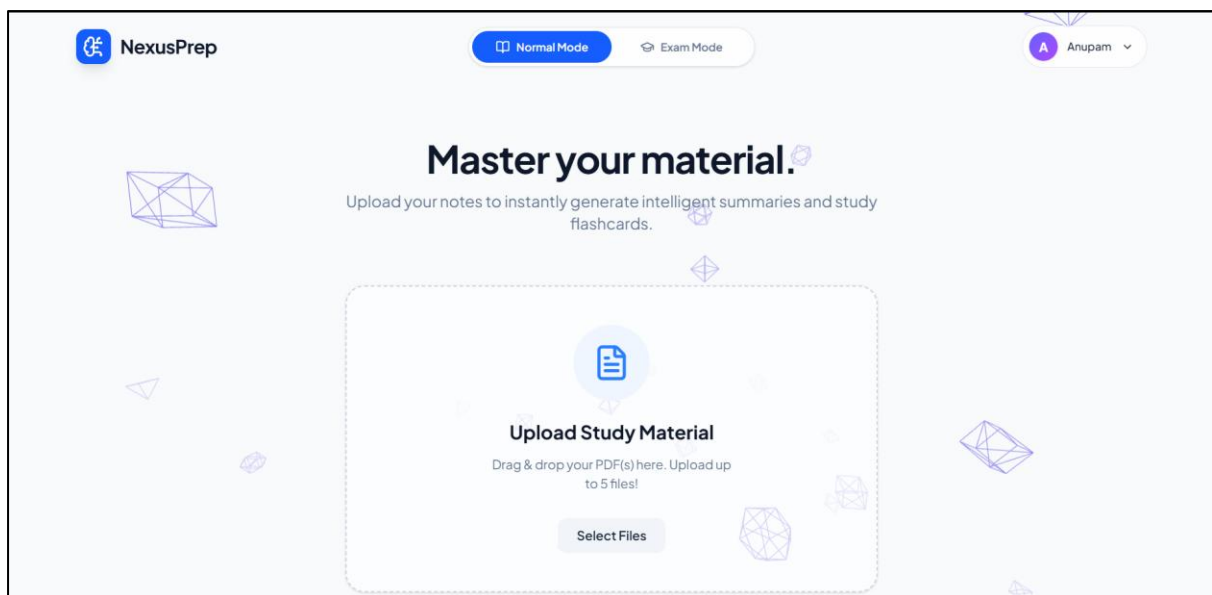


Fig 1.1: NexusPrep Homepage

Architecturally, NexusPrep is organized as a clean two-tier system. The backend, written in Python using the FastAPI framework and served through Uvicorn, is responsible for every piece of business logic in the application: user authentication, document parsing, all

communication with the Gemini API, and all reads and writes to the MongoDB database. The frontend, built in React using Vite as the build tool and styled with Tailwind CSS, is a single-page application responsible entirely for what the user sees and interacts with — rendering results, managing the visual flow between different views (uploading a document, viewing a generated summary, taking a quiz, revising flashcards), and handling client-side concerns such as session persistence and mathematical notation rendering through the KaTeX library.

1.6 Objectives of the Project

The objectives pursued during the design and development of NexusPrep can be summarized as follows:

1. To design and implement a document ingestion pipeline capable of reliably extracting text from both native, digitally-typed PDFs and scanned, image-based PDFs, using an automatic fallback mechanism so the user never needs to know or care which type of document they have uploaded.
2. To integrate a large language model (Google Gemini) into the application in a way that produces consistently structured, well-formatted educational content — summaries, formulas, definitions, and flashcards — while remaining resilient to the model's occasional unavailability or rate-limiting through a fallback system across multiple model versions.
3. To design two functionally distinct study modes — Normal Mode for general comprehension and Exam Mode for targeted examination preparation — each with its own dedicated AI prompting strategy and its own presentation of results.
4. To implement a scientifically grounded, automated spaced-repetition system based on the Ebbinghaus forgetting curve, removing the burden of manually scheduling revision from the student entirely.
5. To build a secure, persistent, cross-device user account system, including email-based OTP verification at registration and password-reset functionality, backed by a cloud-hosted MongoDB database rather than local, browser-based storage.
6. To design a frontend interface that correctly renders complex mathematical notation (formulas, Greek letters, and symbols) using LaTeX-based rendering, since a large proportion of academic material — particularly in STEM subjects — cannot be meaningfully represented in plain text alone.
7. To ensure the application behaves gracefully under partial failure — for example, when a third-party email dispatch genuinely fails, when an AI model is temporarily unreachable,

or when the AI's generated output does not parse cleanly as JSON — rather than allowing such failures to break the user's experience entirely.

8. To gain hands-on, end-to-end experience across the full technology stack required to bring such a system into being, from backend API design through to frontend state management and user experience design.

1.7 Scope of the Project

The scope of NexusPrep, as developed during the course of this vocational training, covers the complete pipeline from document upload to long-term revision scheduling, for an individual student user. Specifically, the project scope includes:

- Document upload and parsing for PDF files only, covering both native-text and scanned/image-based documents, with a per-request limit of five files per upload to keep processing times and AI costs reasonable.
- AI-based content generation for both Normal Mode and Exam Mode, including summaries, formulas (with correct mathematical notation), definitions, flashcards, high-yield topic identification, strategic insights, predicted examination questions, and multiple-choice quizzes.
- A complete user account system, including registration gated behind email OTP verification, login, password reset, and profile management (including optional personalization based on the user's age and education level, which is used to adjust the tone and depth of AI-generated explanations).
- Persistent, cloud-based storage of all flashcard decks, along with an automated spaced-repetition scheduling engine and email-based reminders for overdue revisions.
- A responsive, single-page web frontend supporting the full user journey described above, including an interactive flashcard revision interface and a quiz-taking interface with scoring.

The scope of the project explicitly does not extend to native mobile applications (the platform is web-only), support for non-PDF document formats (such as Word documents or image files uploaded directly), or real-time collaborative study features (such as shared decks between multiple users). These exclusions were deliberate choices made to keep the project achievable within the timeframe of the vocational training while still producing a genuinely complete, usable product rather than a partially finished prototype.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction to the Literature Review

Before describing how NexusPrep was actually built, it is worth stepping back and looking at the body of prior work — both academic and commercial — that the project draws on, whether directly or indirectly. NexusPrep sits at the intersection of three fairly distinct fields: cognitive science research on memory and learning, natural language processing research on document understanding and content generation, and the more practical, engineering-oriented world of full-stack web application design. Very little of what NexusPrep does is conceptually new in isolation — spaced repetition has been studied since the nineteenth century, automated summarization has been an active NLP research area for decades, and the technology stack used (FastAPI, MongoDB, React) is standard in modern web development. What this chapter attempts to do is trace each of these threads individually, and then look at how existing tools have (or have not) combined them, in order to properly situate what NexusPrep contributes.

The chapter is organized into three broad parts. The first part (Sections 2.2–2.4) covers the cognitive science foundation — the forgetting curve, spaced repetition, and active recall — since this is the theoretical basis for the flashcard and revision-scheduling subsystem of NexusPrep. The second part (Sections 2.5–2.8) covers the natural language processing and artificial intelligence background — document parsing and OCR, automated summarization, question generation, and the rise of large language models such as Gemini, which together make it possible to automate what used to require human effort. The third part (Sections 2.9–2.11) surveys existing tools and platforms in the ed-tech space, comparing their approaches against the design NexusPrep eventually adopted, and closes with a discussion of the technology stack literature underpinning the engineering choices made.

2.2 The Forgetting Curve and the Psychology of Memory

The single most important piece of theoretical grounding behind NexusPrep's flashcard and revision system is the work of the German psychologist **Hermann Ebbinghaus**, who in the 1880s conducted a now-famous series of self-experiments on memory retention. Ebbinghaus memorized lists of meaningless syllables and then tested his own recall of them at varying intervals afterward, plotting the resulting data as what has since come to be known as the **forgetting curve**. The central finding was that forgetting is not linear — memory does not

decay at a constant rate over time. Instead, the sharpest drop in retention happens very soon after initial learning, within the first day or two, after which the rate of forgetting slows considerably. A piece of information reviewed only once, without any reinforcement, can lose the majority of its "recallability" within a matter of days.

What makes Ebbinghaus's work directly actionable, rather than purely descriptive, is a second finding that follows from the shape of the curve: if a learner is prompted to recall the same information again *just before* it would otherwise have been forgotten, the resulting memory trace is strengthened, and the next forgetting curve for that same piece of information is flatter — meaning it takes longer before the information needs to be reviewed again. Repeating this process — reviewing information at progressively longer intervals, each one timed to occur just as retention is about to fall off — is what is now generally referred to as **spaced repetition**. Subsequent research over the twentieth and twenty-first centuries has repeatedly validated and refined this basic model. Landmark work by Cepeda, Pashler, Vul, Wixted, and Rohrer, published in *Psychological Bulletin* in 2006, conducted a large-scale meta-analysis of spacing effect studies and confirmed that distributed practice consistently outperforms massed practice (i.e., cramming) for long-term retention, across a wide range of subject matter and learner populations. This body of research is what gives spaced repetition its credibility as a study technique — it is not a folk heuristic, but a well-replicated finding in cognitive psychology.

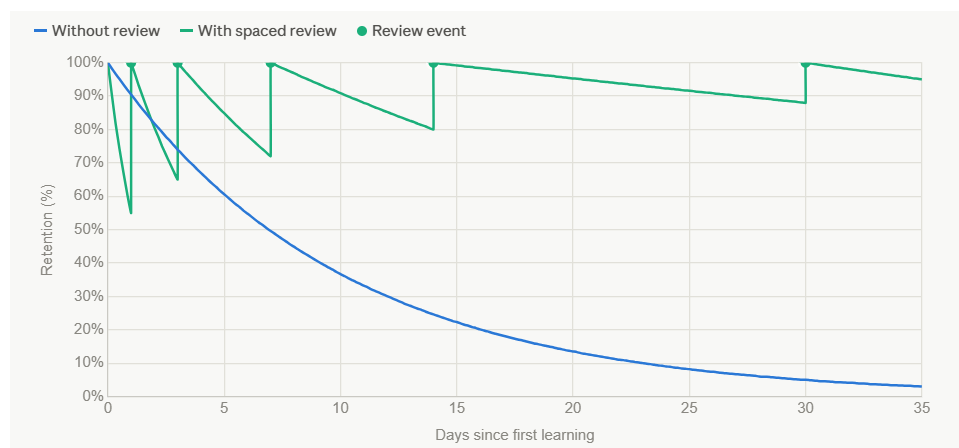


Fig 2.1: The Ebbinghaus forgetting curve

A closely related concept is active recall — the idea that actively retrieving information from memory (for instance, by answering a question) produces stronger, more durable learning than passively re-reading the same material. This is sometimes referred to as the "testing effect," and it has been studied extensively by researchers such as Roediger and Karpicke, whose 2006

study published in *Psychological Science* demonstrated that students who repeatedly tested themselves on material retained it significantly better a week later than students who simply re-studied the same material an equal number of times, even though the re-studying group reported *feeling* more confident about their preparation at the time.

Flashcards, as a study format, sit precisely at the intersection of these two findings — they force active recall (the learner must attempt to answer before seeing the answer) and, when combined with a spaced scheduling system, they apply the spacing effect automatically. This combination is exactly what NexusPrep's flashcard and revision subsystem was designed to operationalize, and the specific interval schedule chosen for the system — 1, 3, 7, 14, and 30 days — is a fairly standard, widely used approximation of an expanding spaced-repetition schedule, similar in spirit to schedules used by long-established spaced-repetition software.

2.3 Spaced Repetition Systems in Practice

While Ebbinghaus's research is nearly a century and a half old, its translation into practical software tools is a much more recent development, and it is worth reviewing briefly since NexusPrep's own revision engine sits in this lineage.

The best-known example of software-based spaced repetition is Anki, an open-source flashcard application originally released in 2006, which implements a variant of the SM-2 algorithm — itself derived from earlier work on the SuperMemo family of spaced-repetition algorithms developed by Piotr Woźniak beginning in the late 1980s. SM-2 and its descendants calculate a per-card "ease factor" that adjusts dynamically based on how difficult the learner reports a card to have been, allowing the interval between reviews to expand or contract on a per-card basis rather than following a single fixed schedule for every card in a deck.

NexusPrep's approach is intentionally simpler than SM-2. Rather than tracking a per-card difficulty rating that adjusts continuously, it applies a single, fixed, expanding interval schedule at the level of the entire session — 1, 3, 7, 14, then 30 days, plateauing after five completed revisions. This is a deliberate design trade-off rather than an oversight: implementing and tuning a full ease-factor-based algorithm like SM-2 is a substantial undertaking in its own right, and given the scope and timeframe of this vocational training project, a simpler, well-established fixed-interval schedule was judged to deliver most of the practical benefit of spaced repetition — namely, moving a student away from cramming and toward distributed review — without the additional complexity of a fully adaptive scheduling engine. This trade-off, and the case for revisiting it in a future version of the system, is discussed further in Chapter 5 (Future Scope).

2.4 Personalization and Learner Context in Educational Technology

A separate strand of educational research relevant to this project concerns the effect of tailoring instructional content to a learner's background — their age, prior knowledge, and educational level. Research in the field of differentiated instruction, a term popularized in K-12 pedagogy by researchers such as Carol Ann Tomlinson, argues that learners absorb material more effectively when its complexity and framing are matched to their existing level of understanding, rather than presented in a one-size-fits-all format.

This principle informed NexusPrep's optional "personal context" feature, in which a user can supply their age and education level, which is then used to adjust the tone, vocabulary, and depth of explanation in AI-generated summaries and definitions. While this is a comparatively lightweight implementation of the differentiated-instruction principle — it adjusts a prompt sent to an AI model rather than implementing any sophisticated learner model — it reflects an attempt to bring at least a basic form of personalization into an otherwise generic content-generation pipeline.

2.5 Document Parsing, OCR, and the Text Extraction Problem

Turning to the more technical side of the literature, one of the foundational problems NexusPrep had to solve is not really an AI problem at all, but a document engineering problem: reliably getting readable text out of an arbitrary PDF file supplied by a student.

PDFs broadly fall into two categories relevant here. The first is native-text PDFs, where the text was originally typed and is stored in the file in a machine-readable form — these can be parsed directly using a PDF library without any need for image analysis. The second is scanned or image-based PDFs, where each page is effectively a photograph or scan of a physical page, with no underlying machine-readable text at all; to a computer, such a page is just a grid of pixels. A very large proportion of real student material — photocopied notes, scanned textbook pages, phone photographs of a whiteboard — falls into this second category, and any tool that ignores it will be unusable for a significant fraction of real-world input.

The standard solution to reading text out of scanned documents is Optical Character Recognition (OCR), a field of computer vision research with a long history, historically dominated by systems such as Tesseract (originally developed at HP and later open-sourced and maintained by Google). Classical OCR engines like Tesseract work by detecting character shapes on a page and matching them against a trained character model, and while highly effective for clean, well-scanned printed text, they historically struggled with handwriting,

unusual fonts, low-quality scans, and complex page layouts such as tables or multi-column text.

A more recent development, and the one NexusPrep actually relies on, is the use of multimodal large language models for OCR-like tasks. Modern models such as Google's Gemini family are trained not just on text but on image-text pairs at massive scale, giving them the ability to "read" an image of a page and transcribe its contents with a level of contextual understanding that classical OCR engines lack — for instance, correctly inferring a partially obscured word from surrounding context, or correctly interpreting a handwritten mathematical symbol based on the equation it appears in. This is the approach NexusPrep uses: rather than depending on a dedicated OCR library, the backend renders each page of an image-based PDF to a high-resolution image and sends it directly to Gemini with an instruction to transcribe the text exactly as it appears, falling back to this path automatically whenever native text extraction yields suspiciously little content (a threshold of fewer than fifty extracted characters is used as the trigger in the current implementation).

For native-text extraction, NexusPrep uses PyMuPDF (distributed under the module name `fitz`), a widely used, actively maintained Python library for reading, rendering, and manipulating PDF documents, built on top of the MuPDF rendering engine. PyMuPDF was chosen over alternative libraries such as `pdfplumber` or `PyPDF2` primarily for its speed and its built-in ability to rasterize pages into images at a controllable resolution — a capability that is directly reused for the OCR fallback path, since the same library that extracts native text is also used to produce the page images sent to Gemini when native extraction fails.

2.6 Automated Text Summarization

Once readable text has been obtained from a document, the next problem is condensing it into something a student can actually study from — a summary, a list of key points, or a set of extracted formulas and definitions. This falls under the long-studied NLP subfield of automatic text summarization.

Historically, summarization research has been divided into two broad approaches. Extractive summarization selects and stitches together the most important existing sentences from a source document, without generating any new text — early systems in this category relied on statistical measures like sentence position, word frequency, and TF-IDF (term frequency–inverse document frequency) scoring to rank sentence importance. Abstractive summarization, by contrast, generates entirely new sentences that capture the meaning of the source material, potentially rephrasing, combining, or simplifying ideas in ways that no single sentence in the

original document does — this is generally considered a harder problem, since it requires the system to genuinely "understand" the content rather than simply rank and select from it.

The rise of the transformer architecture, introduced in the influential 2017 paper "Attention Is All You Need" by Vaswani et al., and its subsequent scaling into large pretrained language models (beginning with models like BERT and GPT, and continuing through to the present generation of models such as GPT-4 and Gemini), effectively resolved much of the historical difficulty with abstractive summarization. These models, trained on vast quantities of text, develop an internal representation of language flexible enough to genuinely paraphrase, restructure, and condense information — capabilities that earlier statistical and even early neural summarization systems could only approximate.

NexusPrep's summary-generation feature — the chapter takeaways, key formulas, and definitions produced in Normal Mode — is built entirely on this class of model, specifically Google's Gemini. Rather than implementing any summarization algorithm from scratch, the system's engineering effort is concentrated on prompt design: carefully instructing the model on the exact structure, tone, and formatting rules the output should follow (for example, the strict LaTeX-formatting conventions described later in Chapter 3), and on reliability engineering around the model's output — validating that the returned content is genuinely well-formed JSON, retrying against fallback models when a call fails, and gracefully handling the cases where the model's output does not perfectly match what was requested.

2.7 Automated Question Generation

A closely related NLP research area, and one directly relevant to NexusPrep's Exam Mode, is automatic question generation (AQG) — the task of producing meaningful, well-formed questions from a body of source text. Early approaches to AQG relied on syntactic transformation rules: identifying a declarative sentence, locating a key entity or fact within it, and mechanically transforming the sentence into a question targeting that fact (for instance, transforming "The mitochondria is the powerhouse of the cell" into "What is the powerhouse of the cell?"). These rule-based systems were reasonably effective for simple factual questions but struggled to produce more sophisticated, higher-order questions — the kind that ask a student to apply, analyze, or synthesize information rather than simply recall an isolated fact.

Large language models have substantially advanced the state of the art here as well, since a model like Gemini can be prompted not just to extract a fact and phrase it as a question, but to reason about *what kind* of question would be pedagogically useful — for instance,

distinguishing between a question worth four marks (typically a short, single-concept question) and one worth eight marks (typically a multi-part or analytical question requiring a more developed answer), a distinction that NexusPrep's Exam Mode explicitly asks the model to make when generating predicted examination questions.

A further relevant strand of research concerns distractor generation for multiple-choice questions — that is, generating incorrect but plausible answer options alongside the correct one. Poorly chosen distractors (ones that are obviously wrong, or absurdly unrelated to the topic) make a multiple-choice question trivially easy regardless of whether the student actually understands the material, undermining the diagnostic value of the quiz. NexusPrep's Exam Mode quiz feature specifically instructs the underlying model to generate deliberately challenging, topic-relevant distractors, an approach made feasible only because the model has a genuine semantic understanding of the subject matter rather than relying on superficial word-matching.

2.8 Large Language Models and the Gemini API

Since NexusPrep's entire content-generation pipeline depends on a single external service — Google's Gemini family of large language models — it is worth reviewing, at a slightly more technical level, what this class of model actually is and why it was chosen over the available alternatives.

Large language models are, at their core, neural networks trained to predict the next unit of text (a "token") given everything that came before it, trained on enormous corpora of text scraped from books, websites, and other sources. What makes models like Gemini or GPT-4 useful for tasks like the ones NexusPrep needs is not this raw next-token prediction ability by itself, but the emergent capabilities that appear once such models are trained at sufficient scale and then further refined through techniques such as instruction tuning and reinforcement learning from human feedback — capabilities including following complex, multi-part instructions, reasoning over long documents, and producing output in a precisely specified structured format (such as JSON matching a particular schema).

This last capability — constrained or structured output generation — is central to how NexusPrep actually uses Gemini. Rather than parsing free-form prose returned by the model and trying to extract structured data from it (a notoriously brittle approach), NexusPrep's backend requests the model's `responseMimeType` be set to `application/json`, instructing the underlying Gemini API to constrain its own output to valid JSON matching an implied schema described in the prompt. Even so, the returned text is parsed using the `json-repair` library rather

than Python's strict built-in JSON parser, since occasional minor formatting slips (a missing comma, an unescaped character) are common enough in AI-generated JSON that a small amount of tolerance meaningfully improves reliability, echoing a broader pattern documented across many production AI-integration engineering write-ups: language model output should always be treated as *probably* well-formed, never *guaranteed* well-formed.

A further characteristic of working with hosted AI APIs, relevant to NexusPrep's design, is their inherent unreliability at the infrastructure level compared to a typical internal service call — rate limits are enforced fairly aggressively on shared endpoints, and individual model versions are occasionally deprecated, degraded, or temporarily unavailable. This is why NexusPrep's backend implements a rolling fallback list of model versions (gemini-2.5-flash, followed by gemini-3.1-flash-lite, followed by gemini-3.5-flash) and an exponential-backoff retry strategy on HTTP 429 (rate-limit) responses — a pattern that mirrors general best-practice guidance published in most major cloud providers' API documentation for working with any rate-limited external service, not something specific to Gemini.

2.9 Comparative Review of Existing Study and Ed-Tech Platforms

No project is built in a vacuum, and it is useful to place NexusPrep alongside existing tools that address overlapping parts of the same problem space, in order to see clearly what already exists and where genuine gaps remain.

Anki, discussed earlier in the context of spaced repetition, remains the dominant dedicated flashcard application, particularly popular among medical students and language learners. Its core strength is a mature, highly configurable spaced-repetition engine (SM-2 and its variants) supported by a large ecosystem of community-made add-ons. Its principal weakness, for the purposes of the problem this project set out to solve, is that Anki does not generate flashcards for the user — a student must still manually author every card, or download a pre-made deck created by someone else, which may not match their specific syllabus or textbook.

Quizlet occupies similar territory to Anki, with a more polished, less technical interface and a strong social/sharing component that lets students find and reuse decks made by others. In more recent years, Quizlet has introduced AI-assisted features that can generate flashcards from pasted text, moving it closer to NexusPrep's Normal Mode. However, it does not offer anything comparable to Exam Mode's cross-referencing of study notes against previous years' question papers, nor does it attempt any subject-matching validation between uploaded materials.

General-purpose AI assistants, such as ChatGPT or Gemini's own consumer-facing chat interface, can in principle be asked to summarize a document or generate flashcards manually,

and many students have started doing exactly this. However, using a general-purpose chat interface for this purpose has real practical limitations compared to a purpose-built tool: the user has to manually craft an effective prompt each time, there is no persistent, organized library of decks with an automatic revision schedule, there is no distinction between comprehension-mode and exam-strategy-mode prompting, and mathematical notation is not reliably or consistently rendered in a readable form across every response. NexusPrep can be understood, in part, as productizing and specializing what a diligent student might otherwise try to replicate manually through a general-purpose chat tool — wrapping the same underlying model capability in a structured, repeatable workflow with persistent storage and purpose-built prompting.

Notion AI and similar productivity-tool integrations bring AI-assisted summarization into a general note-taking environment, but similarly lack any dedicated spaced-repetition engine or exam-specific cross-referencing capability, and are oriented toward general knowledge work rather than academic examination preparation specifically.

Table 2.1: Comparative Overview of Existing Study Platforms vs. NexusPrep

Feature / Capability	Anki	Quizlet	General-Purpose AI Chat Tools	Notion AI	NexusPrep
AI-generated flashcards (from uploaded material)	No	Partial (from pasted text)	Manual, prompt-dependent	No	Yes
Automated spaced-repetition scheduling	Yes (SM-2 based)	Limited	No	No	Yes (Ebbinghaus-based)
Dedicated Exam Mode / PYQ cross-referencing	No	No	No	No	Yes
OCR support for scanned / image-based documents	No	No	Partial (varies by tool)	No	Yes (automatic fallback)
Reliable mathematical notation rendering	Limited (plugin-dependent)	Limited	Inconsistent	No	Yes (KaTeX-based)
Cross-device cloud sync of study data	Yes (paid tier)	Yes (account-based)	No (no persistent study structure)	Yes (general notes only)	Yes (MongoDB-backed)
Subject-matching validation across documents	No	No	No	No	Yes
Automated revision reminders	Partial (app notifications)	No	No	No	Yes (email-based)
Requires manual content authoring	Yes	Partial	Yes (prompt writing)	Yes	No

Note: "Partial" indicates the capability exists in a limited or indirect form.

What this comparison suggests is not that any single feature of NexusPrep is unprecedented, but that the particular *combination* — AI-driven content generation, a genuine dual-mode design separating comprehension study from exam strategy, automatic OCR fallback, an automated spaced-repetition engine, and reliable mathematical notation rendering, all backed by cloud persistence — is not something readily available in a single existing tool aimed at the general student market.

2.10 Literature on the Supporting Technology Stack

Beyond the AI and cognitive-science literature discussed above, a number of more purely engineering-oriented references informed the technology choices made in building NexusPrep, and are worth briefly noting here since they are referenced again in the Methodology chapter.

FastAPI, the Python web framework used for NexusPrep's backend, was chosen largely on the strength of its native support for asynchronous request handling (built on Python's `asyncio`) and its automatic request/response validation through Pydantic models. Its documentation and design philosophy, authored by Sebastián Ramírez, emphasize exactly the two properties most relevant to an AI-integrated backend: the ability to handle many concurrent, slow, I/O-bound operations (such as waiting on a response from the Gemini API) without blocking other requests, and strong, declarative input validation that catches malformed requests before they reach business logic.

MongoDB, used as NexusPrep's persistence layer via the asynchronous motor driver, is a document-oriented NoSQL database, and its suitability here follows directly from the shape of the data being stored. A flashcard deck, for instance, is naturally a nested structure — a deck has a title, a list of cards, and several scheduling-related fields, and the number of cards varies from deck to deck. Representing this in a traditional relational (SQL) database would require a separate join table for cards, whereas MongoDB allows the entire deck, cards included, to be stored as a single JSON-like document, closely matching the JSON structure already being exchanged with the Gemini API and the React frontend — reducing the amount of data-transformation code needed at every layer of the stack.

React, used for the frontend, together with supporting libraries such as KaTeX for LaTeX-based mathematical typesetting and three.js for the ambient background visualization, reflects standard, widely documented practice in modern single-page application development, and is not discussed at length here since the specific implementation choices are covered in full in Chapter 3.

2.11 Summary and Positioning of the Present Work

Drawing the threads of this chapter together: the cognitive science literature establishes that spaced, active recall is a well-validated, effective study technique, but one that is rarely practiced consistently without software support; the NLP and AI literature shows that the technical capability to automatically extract, summarize, and generate questions from documents — including scanned, non-native-text documents — has matured rapidly and is now genuinely accessible through commercial APIs like Gemini, rather than requiring custom model training; and the comparative review of existing tools shows that while individual pieces of this puzzle exist in various products, no single widely available tool combines AI-generated content, a genuine dual-mode (comprehension vs. exam-strategy) design, OCR-based

document flexibility, automated spaced repetition, and reliable mathematical rendering into one coherent system.

NexusPrep's contribution, then, is best understood not as a novel algorithm or a new piece of AI research, but as a carefully engineered synthesis of already-validated ideas from separate fields — psychology, NLP, and full-stack engineering — into a single, usable, end-to-end platform. The remainder of this report, beginning with Chapter 3, describes exactly how that synthesis was carried out in practice.

CHAPTER 3

METHODOLOGY

3.1 Overview of the Development Approach

NexusPrep was not built from a fixed, upfront specification. It evolved iteratively, the way most real software actually gets built — a working skeleton first, then layer after layer of functionality added on top, with earlier decisions occasionally revisited once their limitations became apparent in practice. This chapter documents that process in the order the system's pieces naturally fit together (architecture → data → ingestion → AI generation → rendering → scheduling → security → frontend), rather than strictly in the chronological order each piece was built, since the former gives a much clearer picture of how the finished system actually works.

Broadly, the development followed four overlapping phases:

1. **Architecture and stack selection** — deciding how responsibility would be split between backend and frontend, and choosing the specific frameworks and libraries for each.
2. **Core pipeline construction** — building the document ingestion, AI orchestration, and content generation logic that forms the heart of both Normal Mode and Exam Mode.
3. **Persistence and scheduling** — designing the database schema and building the spaced-repetition engine that turns generated flashcards into a genuine, self-sustaining revision system.
4. **Interface and hardening** — building the React frontend around the above, and then going back through the whole system looking specifically for failure points — a slow AI response, a malformed document, a missing environment variable — and making sure each one degrades gracefully rather than breaking the user's experience.

The remainder of this chapter walks through each of these in turn.

3.2 System Architecture

NexusPrep follows a conventional but deliberately clean two-tier architecture: a Python backend that owns every piece of business logic, and a React frontend that owns everything the user actually sees. This separation was not an arbitrary default — it reflects a specific, considered decision about where different kinds of responsibility belong.

The backend, built with FastAPI and served by Uvicorn, exposes a REST API under the `/api` prefix and is responsible for: user authentication, document parsing (including the OCR fallback path), all communication with the Gemini API, all reads and writes to MongoDB, and the spaced-repetition scheduling logic. Nothing in the frontend talks directly to Gemini or to the database — every single piece of AI-generated content or stored data passes through the backend first. This matters for a fairly practical reason: the Gemini API keys are secrets, and if the frontend called Gemini directly, those keys would have to be embedded in client-side JavaScript, visible to anyone who opened their browser's developer tools. Routing every AI call through the backend keeps the keys server-side, where they belong.

The frontend, built with React and bundled using Vite, is a single-page application. It holds essentially no state beyond the current browser session — a cached copy of the logged-in user's profile — and re-fetches or re-derives everything else (flashcard decks, statistics, generated content) from the backend as needed. This was a deliberate choice made partway through development, after an earlier version of the flashcard feature stored deck data in the browser's `localStorage`. That earlier approach worked, but it meant a student's decks were trapped on whichever single device or browser they'd been created on — logging in from a phone showed nothing, because the phone's browser had never seen those decks. Migrating deck storage to MongoDB, with the frontend as a pure client of that data, was the fix, and is discussed at length in Section 3.7.

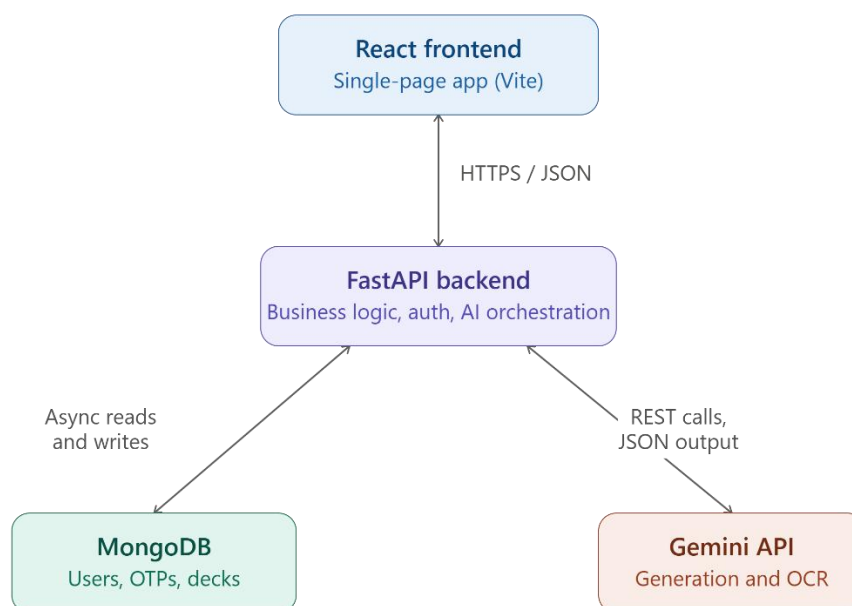


Fig 3.1: NexusPrep Architecture Diagram

Communication between frontend and backend happens entirely over JSON via HTTPS (in local development, plain HTTP on localhost), with file uploads handled as multipart/form-data for the two AI-generation endpoints that accept PDFs. CORS is configured permissively on the backend (`allow_origins=["*"]`) to simplify local development, a point revisited as a limitation in Chapter 5.

3.3 Technology Stack: Choices and Justification

A significant part of the vocational training was not simply *using* a given technology stack, but understanding *why* each specific piece was chosen over its available alternatives. This section documents that reasoning.

3.3.1 FastAPI over Flask or Django

Python offers several mature web frameworks, and Flask in particular is the framework referenced heavily in the sample project reports circulated earlier in this training program. NexusPrep instead uses FastAPI, for two concrete reasons directly tied to the nature of this specific application.

First, FastAPI is built around Python's native `async/await` syntax from the ground up, whereas Flask's synchronous request model would block an entire worker thread for the full duration of a slow operation — and calls to the Gemini API, particularly the heavier structured-generation calls, routinely take several seconds. An `async` framework can service other incoming requests while waiting on a slow AI call to return, rather than leaving the whole application stalled.

Second, FastAPI's tight integration with Pydantic for request validation meant that every endpoint's expected input shape could be declared once, as a typed Python class, and have malformed requests rejected automatically before any business logic ran — visible throughout `main.py` in the form of models like `RegisterModel`, `DeckCreateModel`, and `ProfileUpdateModel`.

3.3.2 MongoDB over a Relational Database

The choice of MongoDB, accessed asynchronously via the motor driver, over a relational option such as PostgreSQL or MySQL, follows directly from the shape of the data NexusPrep needs to store. Consider a single flashcard deck: it has a title, an owning user, a handful of scheduling fields (`revisionCount`, `nextRevisionDate`, `stopRevision`), and a list of cards — and that list can be anywhere from fifteen to twenty items long, with no fixed count. In a relational schema, representing this cleanly would require a separate flashcards table with a foreign key back to a decks table, and every read of a deck would need a join across the two. In MongoDB, the entire deck — cards included — is stored as one JSON-like document, which also happens

to be exactly the shape the data already takes as it moves between the Gemini API response, the FastAPI backend, and the React frontend. Choosing MongoDB removed a whole layer of translation code that would otherwise have been needed to flatten nested AI-generated content into relational tables and reassemble it again on the way out.

3.3.3 Google Gemini over Alternative AI Providers

Several large language model providers could plausibly have powered NexusPrep's content generation — OpenAI's GPT models, Anthropic's Claude models, or Google's Gemini family. Gemini was selected primarily for two reasons relevant to this specific application: its native multimodal support (allowing the same API to handle both text-based content generation and image-based OCR without switching providers or SDKs), and its explicit support for constrained JSON output via the `responseMimeType: application/json` generation config, which is used throughout NexusPrep's structured-content endpoints to avoid the fragility of parsing free-form text responses.

3.3.4 React, Vite, and Tailwind CSS

On the frontend, React was chosen for its component model, which mapped naturally onto NexusPrep's distinct views (authentication, upload, results dashboard, flashcards, quiz, profile). Vite was used as the build tool over the older Create React App tooling for its substantially faster development server start-up and hot-reload times — a genuinely noticeable quality-of-life improvement during iterative UI development. Tailwind CSS was used for styling, allowing UI components to be styled directly in JSX via utility classes rather than maintaining separate stylesheet files, which kept related markup and styling physically close together in the codebase — useful in a single-file frontend architecture like NexusPrep's.

Fig 3.2.1: Import block for App.jsx libraries

```
import React, { useState, useEffect, useRef, useCallback } from 'react';
import * as THREE from 'three';
import {
  BookOpen, GraduationCap, User, FileText, UploadCloud, BrainCircuit,
  Logout, Activity, ChevronDown, ChevronUp, CheckCircle2, ListChecks, Quote, Sparkles,
  AlertCircle, Image as ImageIcon, Target, ArrowRight, XCircle, CheckCircle,
  Timer, RotateCcw, ChevronLeft, ChevronRight, FileQuestion, Star, Lock,
  TrendingUp, Layers, PenTool, Database, Clock, Bell
} from 'lucide-react';
```

Fig 3.2.2: Import block for main.py libraries

```
1  import os
2  import json
3  import fitz
4  import asyncio
5  import aiohttp
6  import random
7  import string
8  import smtplib
9  from email.mime.text import MIMEText
10 from email.mime.multipart import MIMEMultipart
11 from datetime import datetime, timedelta, timezone
12 import google.generativeai as genai
13 from fastapi import FastAPI, UploadFile, File, Form, HTTPException
14 from fastapi.middleware.cors import CORSMiddleware
15 from pydantic import BaseModel
16 from dotenv import load_dotenv
17 from PIL import Image
18 import hashlib
19 import uuid
20 from motor.motor_asyncio import AsyncIOMotorClient
21 import certifi
22 from typing import List, Optional
23 import json_repair
```

3.4 Database Design

NexusPrep's MongoDB database, named `smart_study`, is organized into three collections, each serving a distinct purpose.

The `users` collection stores one document per registered account, holding login credentials (as a SHA-256 password hash, never plaintext), profile fields (name, age, education level), preference toggles (`usePersonalContext`, `storeFlashcards`, `emailReminders`), and a set of lifetime usage statistics (`docsProcessed`, `pyqsAnalyzed`, `strategiesGenerated`, `quizzesTaken`, `totalCardsLearned`, `studyStreak`). Keeping these statistics denormalized directly on the user document, rather than computed on demand from a separate activity log, was a deliberate simplification — the Profile view's statistics tab needs to load quickly, and a single-document read is far cheaper than aggregating across a history collection every time that tab is opened.

The `otps` collection stores transient six-digit verification codes generated during registration, each keyed by email with an associated expiry timestamp set ten minutes after issue. This collection is intentionally treated as disposable — a record is either consumed and deleted upon successful registration, or left to become stale and functionally useless once its expiry passes

(the current implementation does not run a separate cleanup job for expired-but-unused OTPs, a minor point noted again in Chapter 5).

The `flashcard_decks` collection is the largest and most actively used collection, storing one document per deck, each with a generated id (a UUID4 hex string, not MongoDB's own `_id`, so that deck identifiers remain stable and predictable even if a deck is ever migrated or re-inserted), an owning email, a title, the cards array itself, and the four scheduling fields — `revisionCount`, `lastRevised`, `nextRevisionDate`, and `stopRevision` — that drive the spaced-repetition engine described in Section 3.8.

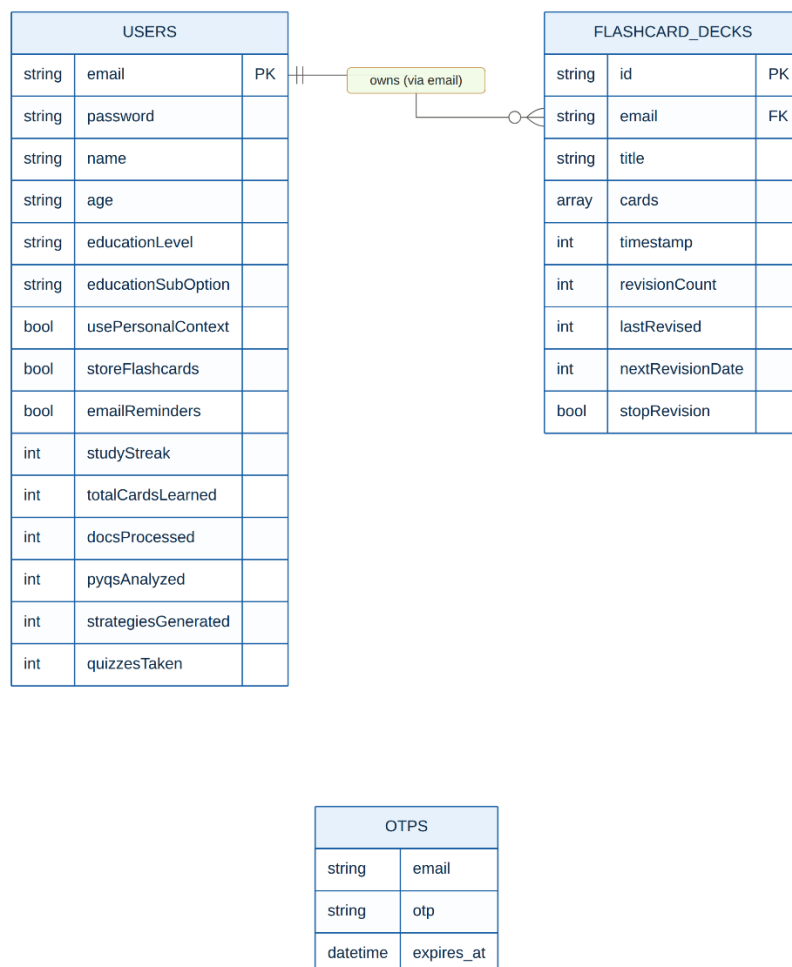


Fig 3.3: NexusPrep entity-relationship diagram

Two design details are worth calling out specifically. First, decks are linked to their owner by email address rather than a separate numeric or ObjectID-based user ID. This was a pragmatic choice — email is already the unique identifier used for login, and avoiding a second identifier removed a class of bugs where a deck's owner reference could silently drift out of sync with

the user's actual account identifier. Second, the `serialize_deck` function applies a backfill step on every deck read: if a deck document is missing the scheduling fields entirely (which happens for decks imported from a user's pre-existing browser `localStorage` via the legacy migration endpoint), those fields are populated with sensible defaults on the fly, rather than requiring every legacy deck to go through a one-time migration script before it can be displayed. This is a small example of a pattern that recurs throughout the backend: prefer handling an inconsistency gracefully at read time over requiring a perfect, pre-cleaned dataset.

3.5 Document Ingestion and the OCR Fallback Pipeline

The very first step in either Normal Mode or Exam Mode is turning an uploaded PDF into plain text the AI model can reason about, and this step turned out to be considerably less trivial than it first appears.

The function `extract_text_with_fallback` implements this logic. It first opens the uploaded PDF using PyMuPDF and extracts whatever native text is embedded in the file, page by page. This works cleanly for the majority of digitally typed documents — a lecture slide exported to PDF, a typed set of notes, an official formula sheet — where the text was never an image to begin with.

The complication is that a large proportion of real student material does not fall into that category. A photocopied textbook chapter, a phone photograph of handwritten notes, or a scanned answer key contains no machine-readable text at all — to PyMuPDF, such a page is just a picture. The system detects this situation using a simple but effective heuristic: if the combined extracted text across all pages of a document comes to fewer than fifty characters, that is treated as strong evidence the document is scanned rather than native, and the pipeline switches to an OCR fallback path.

In the OCR path, each page of the document is rendered to a high-resolution image using PyMuPDF's own rasterization capability (`page.get_pixmap`, scaled by a factor of two to improve legibility for small text), converted into a Pillow Image object, and sent directly to Gemini with an instruction to transcribe the page's text exactly as it appears. This is a meaningfully different approach from a classical OCR engine like Tesseract, discussed in Chapter 2 — rather than matching individual character shapes against a trained font model, a multimodal model like Gemini brings genuine contextual understanding to the transcription, which noticeably improves accuracy on messy handwriting or partially obscured text, where surrounding context can disambiguate an otherwise ambiguous character.

Because OCR calls are comparatively expensive and slow, and because a single multi-page document could otherwise trigger dozens of simultaneous image-analysis requests, this path is protected by a global concurrency limiter — `ocr_semaphore = asyncio.Semaphore(3)` — which caps the number of pages being OCR'd at any one moment, across all concurrent user requests, at three. This was added specifically to prevent a burst of large uploads from overwhelming either the server's own memory (each page image is held in memory during processing) or the Gemini API's own rate limits.

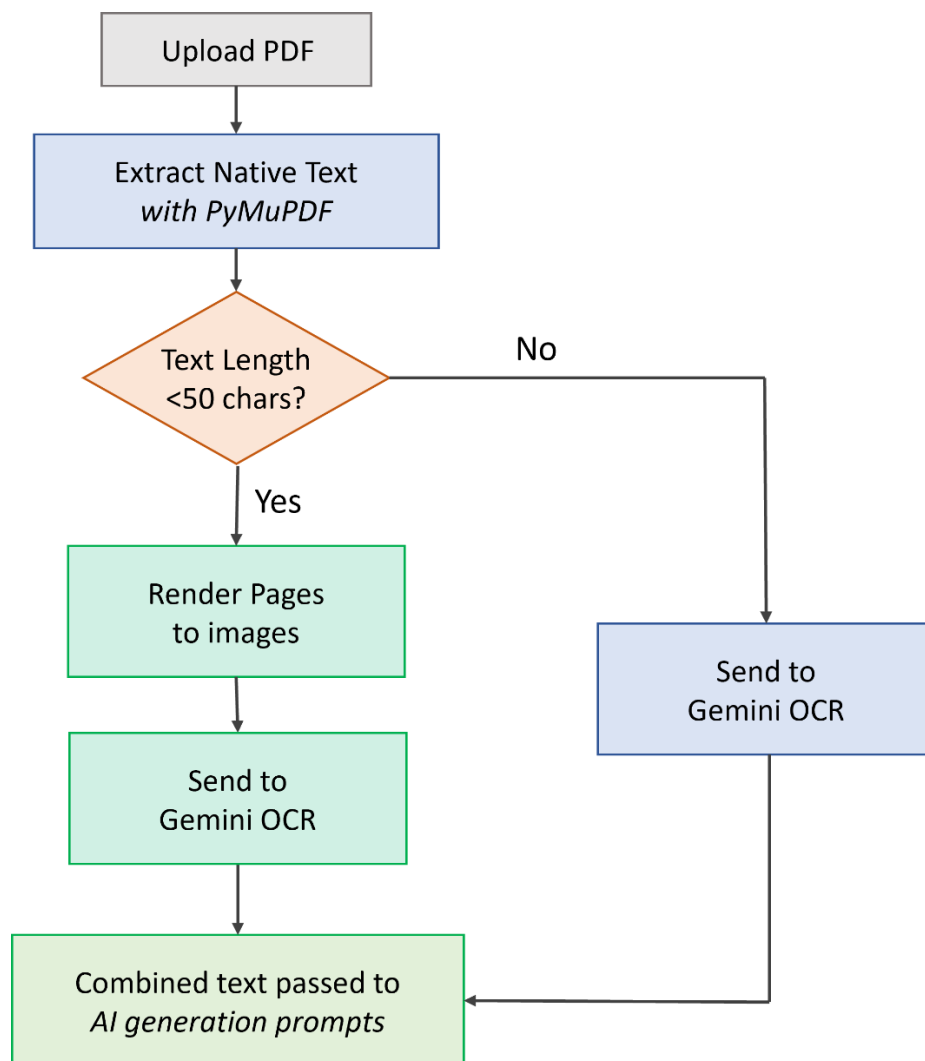


Fig 3.4: PDF processing flow chart

If neither path yields any usable text at all — for instance, a corrupted file, or a document Gemini's OCR genuinely cannot read — the endpoint raises an HTTP 400 error naming the specific offending file, rather than allowing a silent failure to propagate further into the pipeline and produce a confusing empty result downstream.

3.6 AI Orchestration and Prompt Engineering

Once usable text has been extracted, the next stage is turning it into structured educational content, and this is where the bulk of NexusPrep's actual "intelligence" lives — not in a custom-trained model, but in how Gemini is called, prompted, and its output validated.

3.6.1 Two Call Paths, Two Purposes

The backend distinguishes between two distinct kinds of Gemini interaction, each handled through a separate code path.

Lightweight validation and OCR calls go through `call_gemini_with_fallback`, which uses Google's official `google-generativeai` SDK. This path is used for two comparatively simple tasks: checking whether an uploaded document is genuine educational material (and, in Exam Mode, whether the notes and PYQ documents plausibly belong to the same subject), and transcribing OCR page images. If a call fails against one model, this function silently moves on to the next model in the `ACTIVE_MODELS` fallback list before giving up entirely.

Heavy, structured-content generation calls — the ones that actually produce summaries, formulas, flashcards, predicted questions, and quizzes — go through a separate function, `fetch_gemini_json`, which bypasses the SDK entirely and makes raw HTTP calls directly against Gemini's REST endpoint using `aiohttp`. This function explicitly sets `generationConfig.responseMimeType` to `"application/json"`, instructing Gemini's own decoding process to constrain its output to valid JSON, rather than relying on prompt instructions alone (asking a model nicely to "return only JSON" is far less reliable than actually constraining its output format at the API level).

3.6.2 Retry and Fallback Logic

Both call paths were built with the explicit assumption that any single call to an external AI API can fail, for reasons entirely outside the application's control — a temporarily overloaded model, a deprecated model version, or a burst rate limit on a shared quota. `fetch_gemini_json` handles an HTTP 429 (rate-limit) response with exponential backoff, waiting $2^i + \text{random jitter}$ seconds before retrying, for up to five attempts, before giving up on that particular model and moving to the next one in the fallback array. A 404 (typically indicating a deprecated or unavailable model name) or any other non-200 response causes an immediate move to the next model rather than a wasted retry loop. Only once every model in `ACTIVE_MODELS` has been exhausted does the function finally raise an HTTP 500 back to the frontend.

This fallback array — `["gemini-2.5-flash", "gemini-3.1-flash-lite", "gemini-3.5-flash"]` — was one of the more instructive parts of the training, precisely because it illustrates a lesson that isn't obvious until you've actually built something against a hosted AI API: model availability

is not static. A model that works perfectly today can be deprecated, rate-limited more aggressively, or occasionally slow, without warning, and a production-minded system has to plan for that rather than hard-coding a single model name and hoping for the best.

3.6.3 Prompt Design for Normal Mode and Exam Mode

The actual prompts sent to Gemini were iterated on considerably over the course of development, and their final form reflects a number of specific, hard-won lessons.

For Normal Mode, triggered via POST /api/summarize, two prompts are dispatched concurrently: one requesting a short subject title, a set of summary points, and extracted formulas; a second, separate prompt requesting fifteen to twenty flashcard-ready term/definition pairs. These are deliberately split into two separate API calls rather than one combined request — partly to keep each individual prompt focused and easier for the model to follow precisely, and partly because splitting the work across two calls (dispatched under the two separate API keys, `api_key_1` and `api_key_2`) spreads load across two separate rate-limit quotas, improving overall throughput. The two calls are staggered by half a second (`await asyncio.sleep(0.5)`) before being dispatched, a small but deliberate measure to avoid firing an identical burst of requests in the same instant, which was found during testing to trigger rate limits more readily than the same volume of requests spread over even a very short interval.

For Exam Mode, triggered via POST /api/exam-strategy, the same pattern is extended across three concurrent prompts: one for high-yield topics, strategic insights, formulas, and the difficult MCQ quiz; a second, entirely separate prompt specifically for predicted descriptive questions (explicitly instructed to produce only 4-mark and 8-mark style questions, since these are the two question weightings most commonly seen in the university's own examination pattern); and a third for flashcard definitions, mirroring Normal Mode's approach.

Before either mode's main generation prompts are even dispatched, a lightweight validation call checks that the uploaded material is genuine educational content and, in Exam Mode specifically, that the notes and PYQ documents plausibly belong to the same subject — rejecting the request early with a clear HTTP 400 error if not, rather than allowing mismatched or nonsensical input to reach the more expensive generation calls and produce a low-quality or incoherent result.

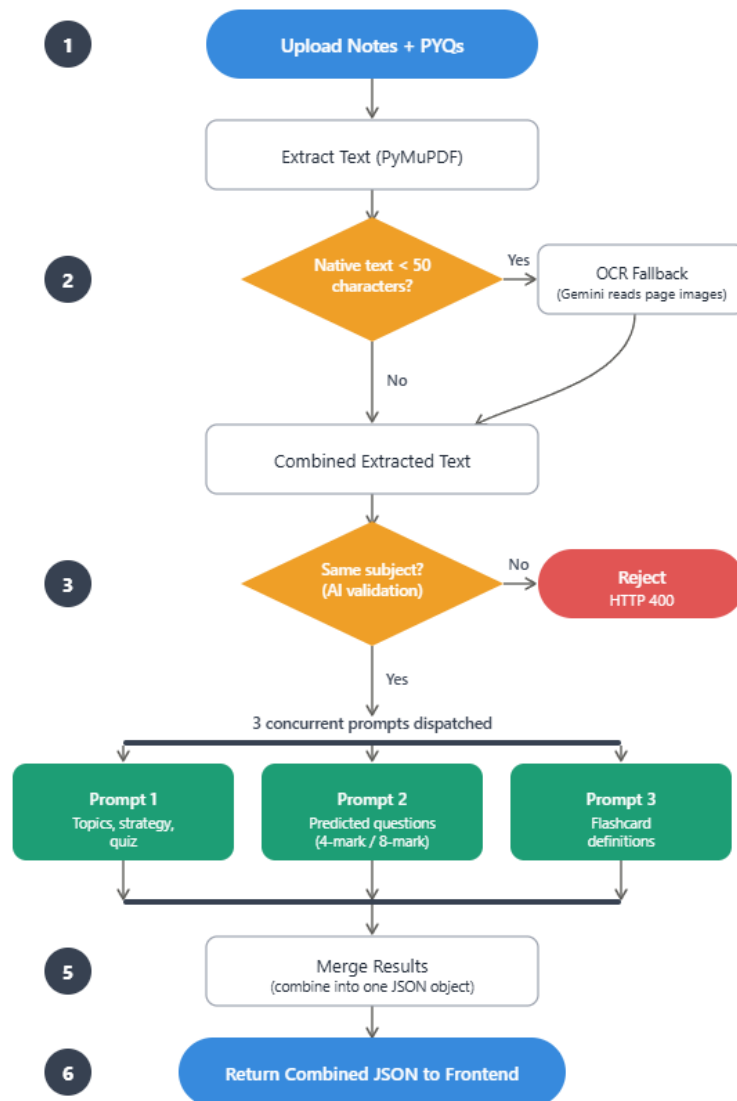


Fig 3.5: Exam Mode request lifecycle flowchart

3.6.4 Handling Imperfect AI Output

A recurring lesson from this part of the project is that AI-generated text, even when the API is explicitly asked to return JSON, cannot be trusted to be perfectly well-formed one hundred percent of the time — an unescaped quotation mark inside a generated explanation, or a missing comma, is an occasional but real occurrence. Rather than parsing the model's raw response text with Python's strict built-in `json.loads`, which would raise an exception and fail the entire request on any minor formatting slip, `fetch_gemini_json` uses the third-party `json_repair` library, which tolerantly repairs common formatting issues before parsing. If the response still cannot be salvaged into a usable dictionary, the function returns an empty dictionary rather than raising an exception, allowing the calling endpoint to substitute reasonable defaults (an

empty list of definitions, for instance) rather than failing the user's entire request over one malformed section of the AI's output.

3.7 Mathematical Notation Rendering Pipeline

A large share of the study material NexusPrep is actually used on — formula sheets, engineering mathematics, physics derivations — is unreadable without correctly rendered mathematical notation. A summary point that reads "the derivative of x squared is $2x$ " is a poor substitute for the same statement rendered as an actual equation, and this became one of the more technically fiddly parts of the entire project to get right, precisely because it sits at the boundary between what the AI model generates and what the browser actually displays.

The approach taken has two halves: strict rules on the generation side, and defensive handling on the rendering side.

On the generation side, every single content-generation prompt sent to Gemini includes a shared block of instructions, `MATH_FORMATTING_RULES`, which spells out a precise convention: every mathematical symbol, formula, or Greek letter in prose fields must be wrapped in `$...$` (inline) or `$$...$$` (block) delimiters; Greek letters must be written as LaTeX commands (`\rho`) rather than spelled out ("`rho`"); and LaTeX commands must use a single backslash and single curly braces, exactly as they would appear in a real `.tex` file, rather than being pre-escaped by the model in anticipation of JSON encoding (since Gemini's own constrained JSON decoding already handles that escaping automatically — a subtlety that, when it was first missed, produced formulas rendered with visibly doubled backslashes on screen).

On the rendering side, the frontend uses KaTeX, loaded dynamically from a CDN rather than bundled as an npm dependency, to actually typeset this LaTeX notation into visually correct mathematics. Two React components carry this work: `MathRenderer`, which renders a single inline or block math expression, and `FormattedText`, which scans an arbitrary block of AI-generated prose, splits it on `$...$` / `$$...$$` boundaries, and routes each mathematical segment through `MathRenderer` while leaving ordinary prose untouched.

Even with careful prompt discipline, the system was built to tolerate occasional slips gracefully rather than assume the AI will always follow the rules perfectly. `MathRenderer` strips leading and trailing `$` or `$$` independently rather than requiring a perfectly symmetric match, so an asymmetric formatting slip (a trailing `$$` with no matching opening delimiter) still renders sensibly instead of breaking. `FormattedText` similarly strips any leftover, unmatched `$$`

sequences that didn't pair up cleanly during splitting, so a stray delimiter degrades to simply disappearing from view rather than leaking onto the screen as a literal, confusing dollar sign.

3.8 The Spaced Repetition Engine

The theoretical basis for this subsystem was covered in Chapter 2; this section documents how it was actually implemented.

Every flashcard deck generated by either Normal or Exam Mode, once saved, is scheduled for revision using a fixed, expanding interval sequence: `REVISION_INTERVALS_DAYS = [1, 3, 7, 14, 30]`. When a deck is first created, its `nextRevisionDate` is set to exactly one day after creation. Each time a user completes a full revision session on that deck — flipping through every card — the backend's `PUT /api/flashcards/decks/{deck_id}/revision` endpoint is called, which increments the deck's `revisionCount`, looks up the next interval via `get_next_revision_interval(revision_count)` (which simply indexes into the interval array, clamped so that a `revisionCount` of five or more always resolves to the final thirty-day interval), and sets a new `nextRevisionDate` that many days out from the current server time.

Deliberately, this scheduling logic lives entirely on the server side rather than being computed in the browser. This was not an arbitrary choice — if the "next revision due" calculation lived in client-side JavaScript, a user switching between a laptop and a phone could see two different, potentially inconsistent revision dates depending on which device last performed the calculation, and the two would have no reliable way to reconcile. Computing it once, server-side, at the moment a revision is actually completed guarantees a single, consistent source of truth regardless of which device the user is using next.

Detecting *overdue* decks — ones whose `nextRevisionDate` has already passed — happens on the frontend at login time: the app fetches the user's full deck list and filters for any deck where `nextRevisionDate` is in the past and `stopRevision` is not set. Any overdue decks are surfaced as an in-app notification, and, provided the user has not opted out via their `emailReminders` preference, an email reminder is dispatched through `POST /api/reminders/dispatch`. This reminder dispatch is explicitly throttled to at most once every twenty-four hours per user — implemented as one of the very few pieces of logic in the entire application that deliberately still lives in browser `localStorage` (a timestamp of the last reminder sent), since this specific value is a UI-level rate limit rather than actual study data, and does not need to be consistent across devices the way deck content does.

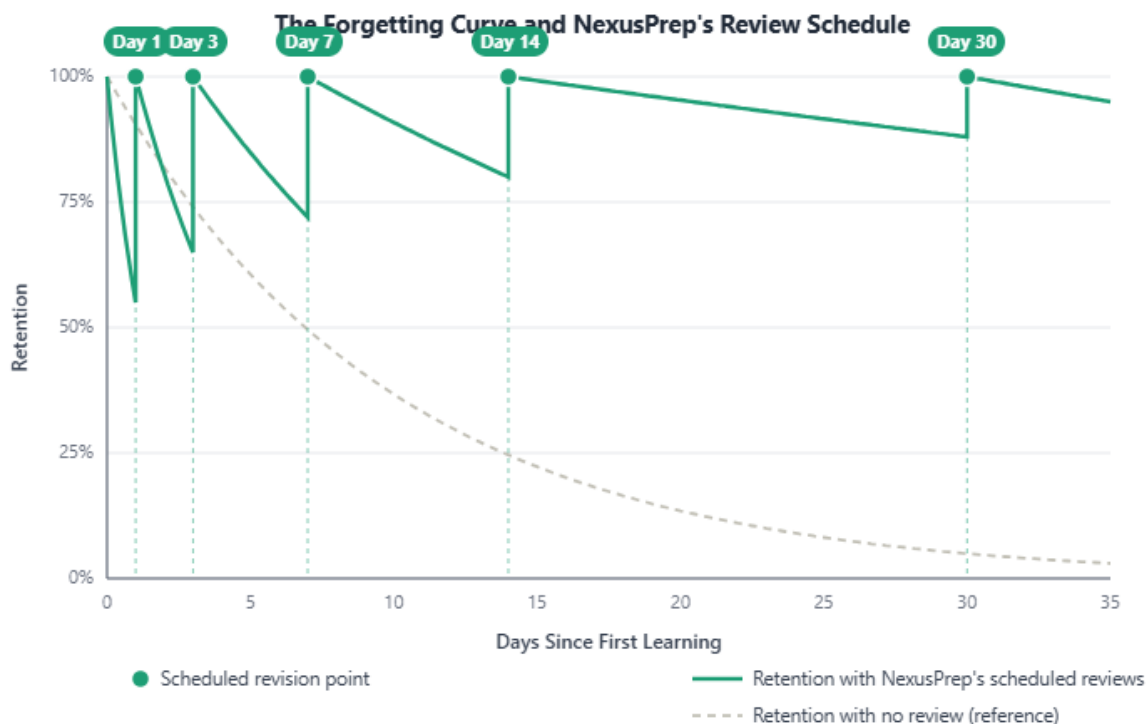


Fig 3.6: Ebbinghaus curve with NexusPrep's five scheduled review points

A user can also opt a single deck out of future reminders individually, without affecting their global emailReminders preference or any other deck, via a dedicated per-deck stopRevision flag — a small but meaningful distinction, since a student who has simply finished with one subject's deck should not have to disable reminders for every other deck they are still actively revising.

3.9 Authentication, Security, and Session Handling

User registration is gated behind email verification: a user first requests a one-time six-digit code via `POST /api/auth/send-otp`, which is generated, stored in the `otps` collection with a ten-minute expiry, and mailed via Brevo's API. Registration itself, via `POST /api/auth/register`, validates the submitted code against this stored record — checking both that the code matches and that it has not expired — before creating the actual user document, at which point the consumed OTP record is deleted so it cannot be reused. Passwords are never stored in plaintext; they are hashed with SHA-256 before being written to the database, and login compares hashes rather than raw passwords.

Password reset, via `POST /api/auth/reset`, does not require the user to prove ownership of the account beyond knowing their own email address — it generates a new, randomly constructed strong temporary password (deliberately including at least one uppercase letter, one lowercase

letter, one digit, and one special character, then shuffled to avoid a predictable pattern) and emails it directly, after which the user is expected to log in and change it. This is a reasonably common, low-friction pattern for smaller applications, though it does mean anyone with access to a user's email inbox could reset that user's NexusPrep password — a limitation acknowledged and revisited in Chapter 5 alongside other security hardening ideas (such as a proper reset-link/token flow) that fell outside this training's scope.

On the frontend, session persistence depends on whether the user opted to "stay signed in" at login: if so, the logged-in user's profile is cached in `localStorage`, surviving a full browser restart; if not, it is cached in `sessionStorage` instead, cleared automatically when the tab closes, and additionally paired with a thirty-minute inactivity auto-logout timer that clears the session if no user interaction is detected in that window — a reasonable middle ground between convenience and not leaving an account logged in indefinitely on a shared or public computer.

3.10 Frontend Design and User Flow

The frontend is a single-file React application (`App.jsx`) organized around a set of clearly delineated view components, each corresponding to a distinct stage of the user's journey through the app: an `AuthView` handling login, OTP-gated registration, and password reset; a `HomeView` and `ResultsDashboardView` for Normal Mode's upload flow and results; an `ExamModeView` and its accompanying results dashboards for Exam Mode's dual-upload flow; a `QuizView` for interactive scoring; a `FlashcardsView` implementing the flip-card revision interface; and a `ProfileView` housing account settings, usage statistics, and the cloud-backed "Saved Decks" library.

A few interface decisions are worth calling out specifically, since they reflect deliberate usability choices rather than defaults. The personalization feature, where a user can optionally supply their age and education level, feeds directly into `construct_personal_context` on the backend, which injects a short instruction (e.g., "user is a 19-year-old studying undergraduate engineering; tailor vocabulary and complexity accordingly") into every generation prompt — meaning the exact same source document can, in principle, produce a differently pitched explanation depending on who is studying it, a lightweight but genuine implementation of the differentiated-instruction principle discussed in Chapter 2.

The ambient animated background, built using `three.js`, was included purely as a polish detail — a subtle, pausable particle/geometry animation intended to make the interface feel less like a bare utility and more like a considered product, without interfering with readability or

performance on lower-end devices (it can be paused, and does not run during the more content-dense revision and quiz views).

NexusPrep Anupam

Trees and Graph Algorithms

[View Documents](#) [Upload New File\(s\)](#)

[Start Quiz](#) [Flashcards](#)

Key Takeaways

- A tree is a connected graph that contains no circuits, meaning there is exactly one path between any pair of vertices.
- A tree with n vertices always has $n - 1$ edges and is considered a minimally connected graph.
- Every tree with two or more vertices must have at least two pendant vertices (vertices with degree one).
- The distance $d(v_i, v_j)$ between two vertices is the length of the shortest path between them, and this distance satisfies the properties of a metric.
- The eccentricity $E(v)$ of a vertex v is the maximum distance from v to any other vertex in the graph; a center of a graph is a vertex with minimum eccentricity. Trees have either one or two centers.
- A rooted tree has a designated root vertex. A binary tree is a rooted tree where the root has degree two, and all other vertices have a degree of one or three.
- A spanning tree of a connected graph G is a subgraph that includes all vertices of G and is itself a tree.

Fig 3.7.1: Normal Mode Results dashboard with Key Takeaways, View Documents, and the flashcard/quiz launch option

NexusPrep Anupam

Key Formulas

Tree Edges

$$E = N - 1$$

The number of edges (E) in a tree is one less than the number of vertices (N). This property holds for any tree.

Binary Tree Edges (as stated in text)

$$e = \frac{1}{2} [1 \times 1 + (n - p - 1) \times 3 + p \times 1]$$

The formula for the number of edges (e) in a binary tree, where n is the total number of vertices and p is the number of pendant vertices. This formula is presented in the text.

Minimum Binary Tree Height

Fig 3.7.2: Normal Mode Results dashboard with generated Key Formulas

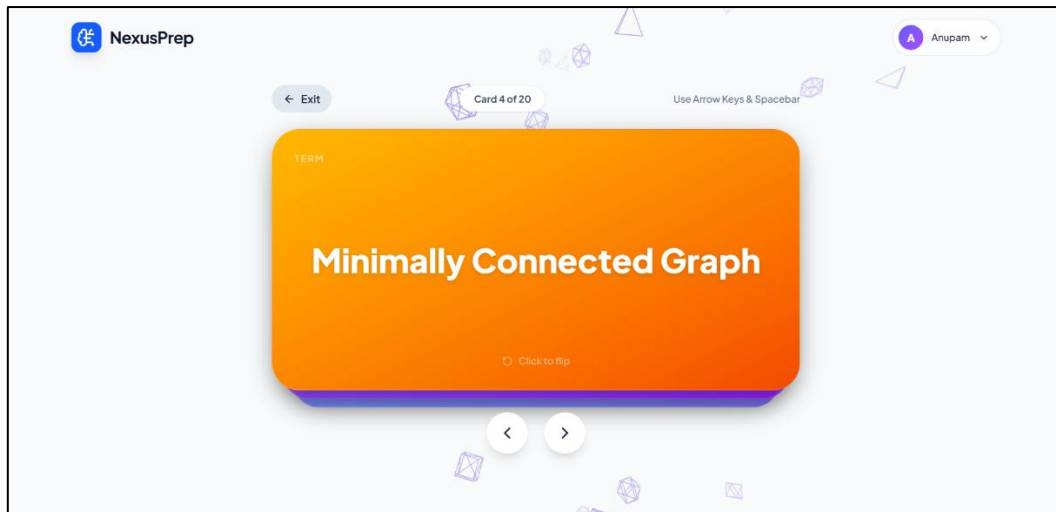


Fig 3.8.1: Flashcard revision interface with an un-flipped card showing card title

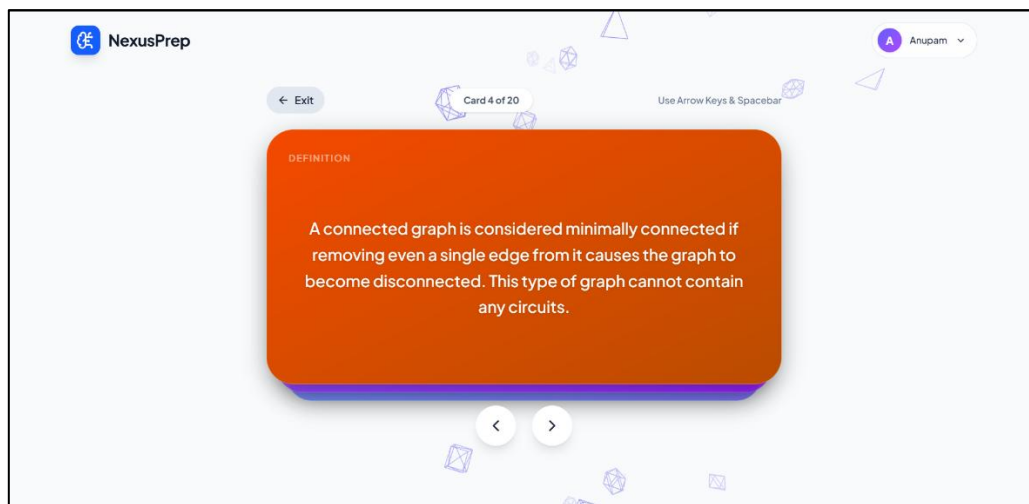


Fig 3.8.3: Flashcard revision interface with a flipped card showing definition

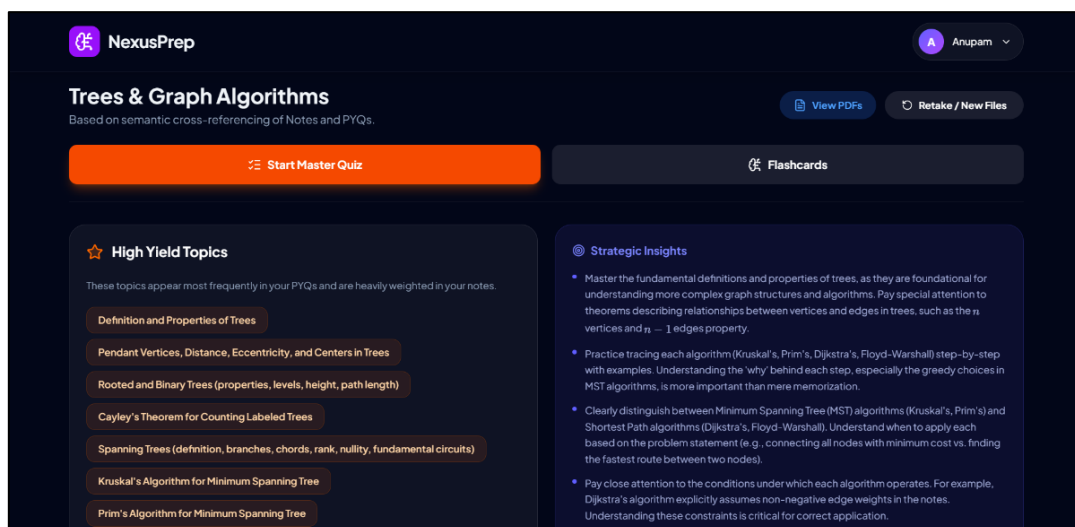


Fig 3.9.1: Exam mode results dashboard showing yield topics and strategic insights

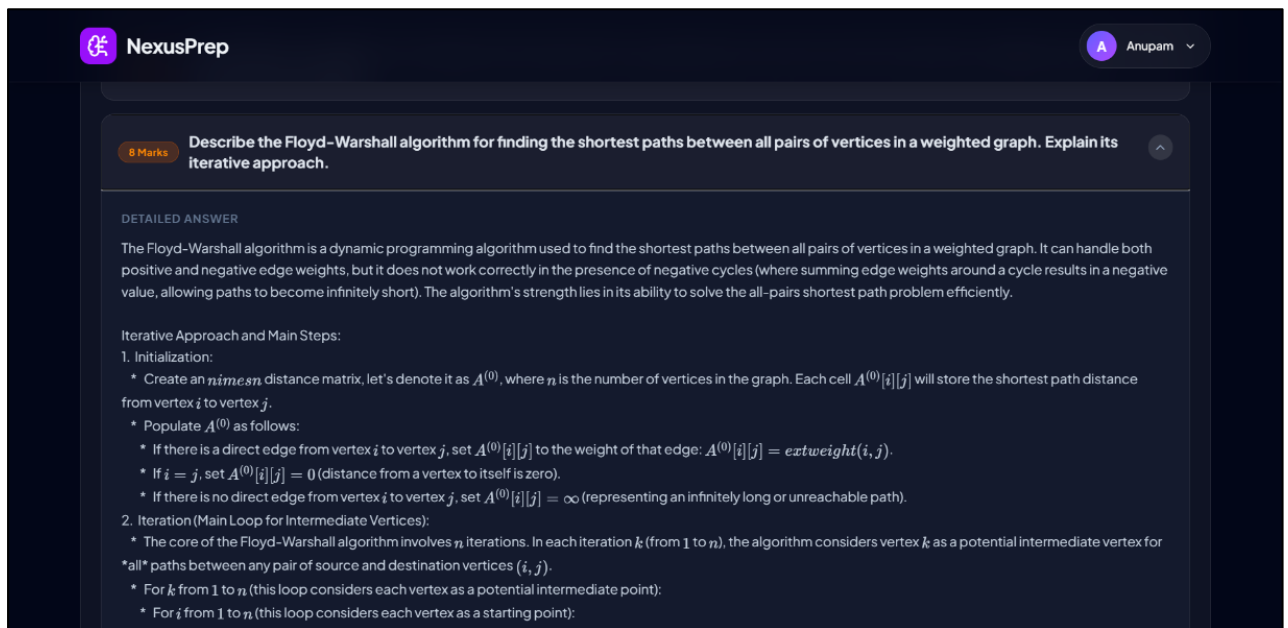


Fig 3.9.2: Exam mode results dashboard showing predicted questions.

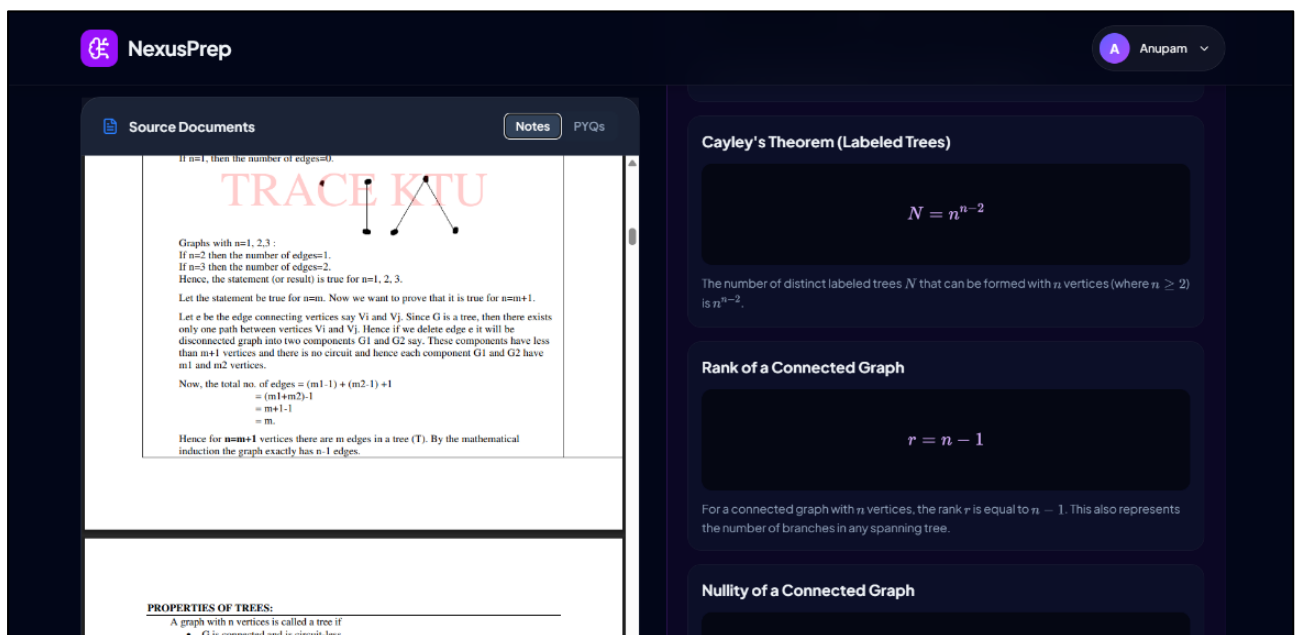


Fig 3.9.3: Split Screen interface to access course material

3.11 API Design Summary

For reference, Table 3.1 summarizes the complete set of backend endpoints exposed by NexusPrep, grouped by functional area.

Method	Endpoint	Purpose
POST	/api/auth/send-otp	Generates and emails a 6-digit OTP for registration
POST	/api/auth/register	Validates the OTP and creates a new user account
POST	/api/auth/login	Authenticates a user and returns their profile
POST	/api/auth/reset	Generates and emails a new temporary password
PUT	/api/profile/update	Updates profile fields and preference toggles
POST	/api/stats/increment	Increments a study statistic counter
POST	/api/reminders/dispatch	Sends an email revision reminder
GET	/api/flashcards/decks	Fetches all flashcard decks owned by a user
POST	/api/flashcards/decks	Creates a new flashcard deck
POST	/api/flashcards/decks/import	Bulk-imports legacy local decks into MongoDB
PUT	/api/flashcards/decks/{id}/revision	Marks a deck revision complete and reschedules it
PUT	/api/flashcards/decks/{id}/stop	Disables future reminders for a deck
DELETE	/api/flashcards/decks/{id}	Deletes a flashcard deck
POST	/api/summarize	Generates Normal Mode summary, formulas, and flashcards
POST	/api/exam-strategy	Generates Exam Mode strategy, questions, and flashcards

Table 3.1: NexusPrep API Endpoint Reference

3.12 Testing and Validation Approach

Given the scope and duration of the vocational training, formal automated test suites (unit tests, integration tests run through a CI pipeline) were not built out for NexusPrep; testing instead took the form of structured manual verification against a defined checklist, repeated after every significant change to the ingestion or generation pipeline. This checklist covered: uploading a native-text PDF and confirming a coherent summary and correctly rendered formulas; uploading a deliberately low-quality scanned image to confirm the OCR fallback triggered correctly; uploading mismatched Notes and PYQ documents in Exam Mode to confirm the subject-validation check correctly rejected the request; completing a full flashcard revision cycle and confirming the `nextRevisionDate` advanced by the expected interval; and logging into the same account from two different browsers to confirm decks and statistics stayed correctly synchronized through MongoDB rather than diverging.

This manual approach, while less rigorous than an automated test suite, was judged appropriate for the scope of the training, and is explicitly flagged in Chapter 5 as one of the clearest opportunities for future improvement were the project to be developed further.

3.13 Summary of the Methodology

Taken together, the methodology described in this chapter reflects a system built less around any single clever technique and more around a large number of individually modest, carefully considered engineering decisions — where AI orchestration belongs, how to survive an unreliable third-party API, how to represent nested data cleanly, how to schedule revisions consistently across devices, and how to keep the interface working sensibly even when one of its many moving parts briefly misbehaves. The following chapter, Results and Discussion, evaluates how well this design actually performs in practice.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Overview of the Evaluation Approach

NexusPrep is not a predictive model in the conventional sense — there is no single accuracy number, comparable to a MAPE or an R^2 score, that summarizes how well it performs, because the system does not predict a numeric outcome from historical data. What it does instead is orchestrate a large language model to transform unstructured documents into structured study material, and the meaningful way to evaluate that is by examining the *functional correctness* of the output (does it produce a coherent, usable summary, deck, or strategy from real input?), the *reliability* of the pipeline under realistic conditions (does it survive a slow API response, a scanned document, a malformed model output?), and the *practical usability* of the resulting interface.

Given the vocational, single-developer, time-boxed nature of this project, evaluation was carried out through the structured manual testing process described in Section 3.12, run repeatedly across a set of real study materials — engineering mathematics notes, a set of previous years' question papers from a university course, and several scanned photocopy chapters of varying scan quality — rather than through a formal, statistically powered benchmark. The results presented in this chapter are the outcomes of that testing process, and are reported with that context clearly attached rather than presented as more rigorous than they are.

4.2 Functional Results: Normal Mode

Normal Mode was tested against a range of native-text PDFs spanning several subject areas — a set of typed physics notes, a formula-heavy mathematics chapter, and a conceptual, prose-heavy chapter from a humanities-adjacent subject — in order to check that the pipeline behaved sensibly across genuinely different kinds of source material, rather than only the one subject area it happened to be developed against.

Across these tests, the system consistently produced:

- A short, accurate subject title correctly identifying the broad topic of the uploaded material, even when the document itself never explicitly stated a title.

- A set of chapter-wise summary takeaways that captured the substance of the source material in noticeably more concise form, without dropping any concept that a quick manual read-through of the source flagged as important.
- Correctly identified and LaTeX-formatted formulas for the mathematics-heavy input, rendering cleanly through KaTeX on the frontend, including fractions, subscripts, and Greek letters.
- A deck of flashcards within the expected fifteen-to-twenty range, with term/definition pairs that, on manual inspection, accurately reflected the source content rather than reading as generic or hallucinated filler.

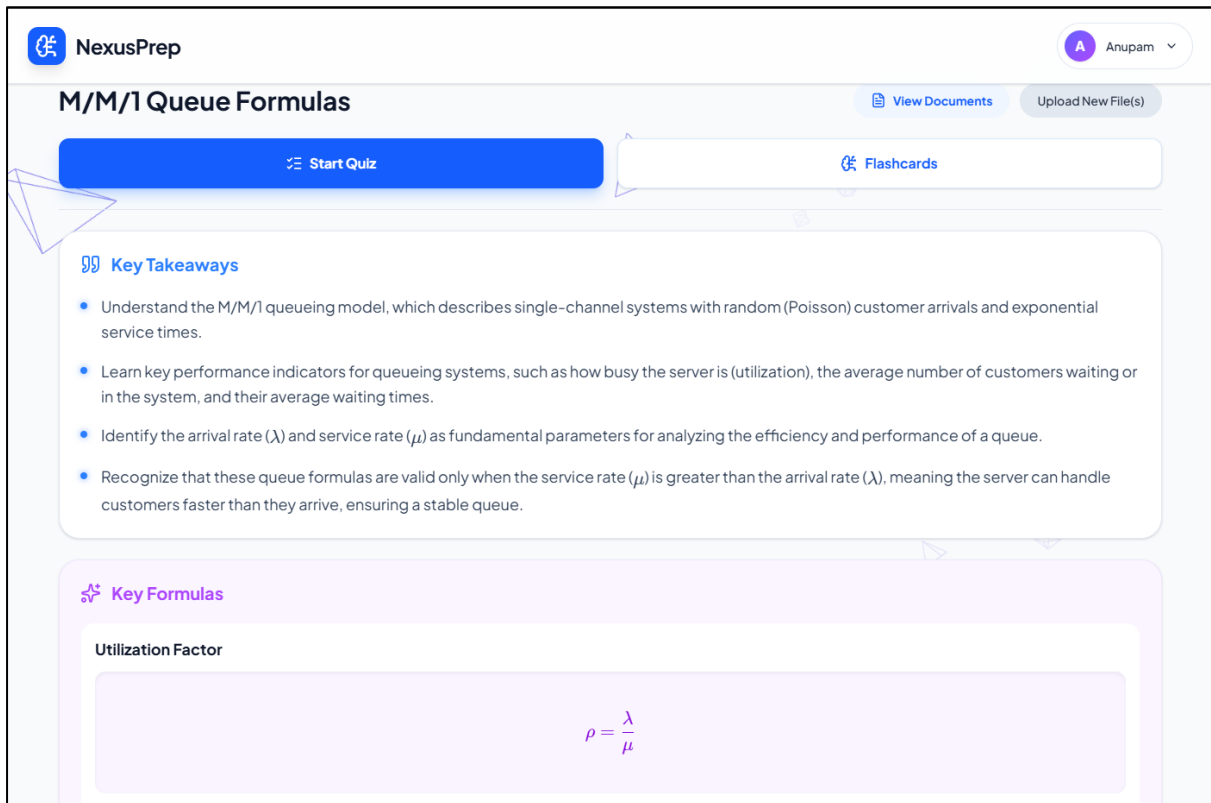


Fig 4.1: ResultsDashboardView after processing a Normal Mode document

One recurring, useful observation from this testing was that summary quality was noticeably better on well-structured source documents (clear headings, one concept per paragraph) than on dense, unstructured prose. This is not a flaw specific to NexusPrep so much as a general characteristic of how language models process text — a well-organized input gives the model clearer signal about where one idea ends and the next begins, which is a genuinely useful thing to know when advising future users on what kind of material to upload for the cleanest results, and is discussed further as a design consideration in Section 4.9.

4.3 Functional Results: Exam Mode

Exam Mode testing followed the same pattern but naturally involved two documents per test run — a notes file and a corresponding PYQ file — plus an additional layer of testing specifically targeting the subject-matching validation step, since this is the one part of Exam Mode with no equivalent in Normal Mode.

Subject-validation behavior. Uploading genuinely matched Notes and PYQ documents (both drawn from the same university subject) reliably passed validation and proceeded to full generation. Deliberately uploading mismatched pairs — for instance, a set of physics notes alongside a computer networks question paper — was correctly and consistently rejected with an HTTP 400 response before either of the three expensive generation prompts was ever dispatched, confirming that the validation step functions as intended as a gatekeeping mechanism rather than a purely cosmetic check.

Content-generation results. For correctly matched document pairs, the system reliably produced:

- A list of high-yield topics, generally aligning well with topics a manual read-through of the PYQ document would independently flag as frequently recurring.
- Strategic insights phrased as genuinely actionable guidance (e.g., prioritizing a specific unit given its repeated appearance across multiple years' papers) rather than generic, one-size-fits-all advice.
- More than ten predicted descriptive questions, correctly split between 4-mark and 8-mark framing, with the 8-mark questions consistently reflecting a broader, more multi-part structure than the 4-mark ones — indicating the model was genuinely differentiating by expected answer depth rather than just varying question length superficially.
- A multiple-choice quiz where, on manual review, the distractor options were topically relevant and plausible rather than trivially eliminable, meeting the "deliberately challenging" design goal set out in Section 3.6.3.

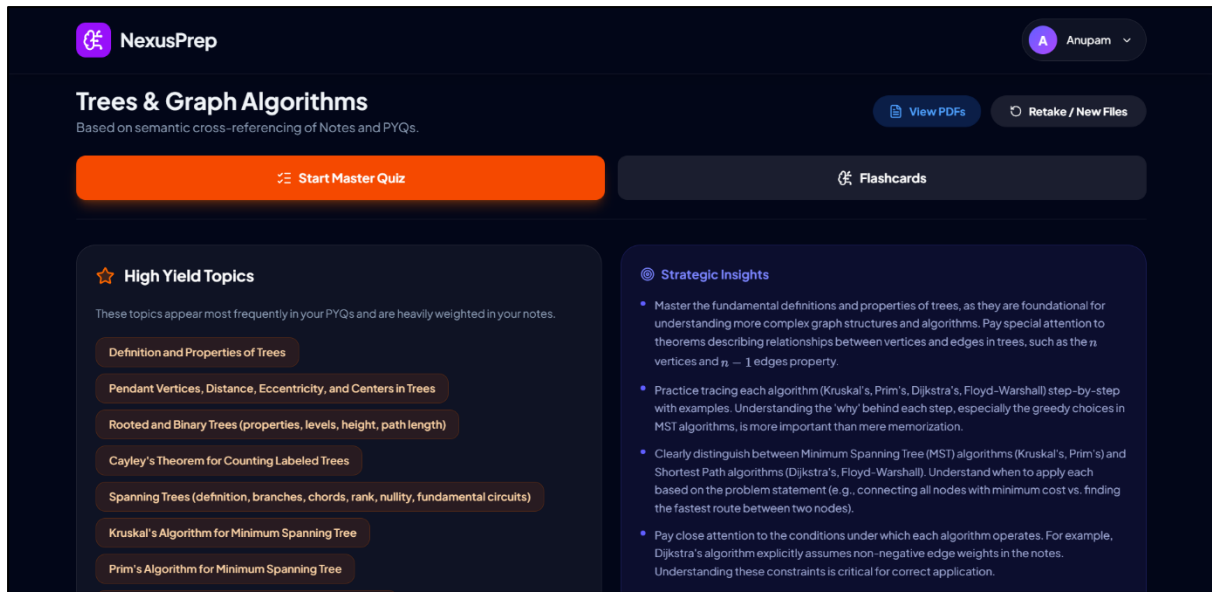


Fig 4.2: Exam Mode Results dashboard

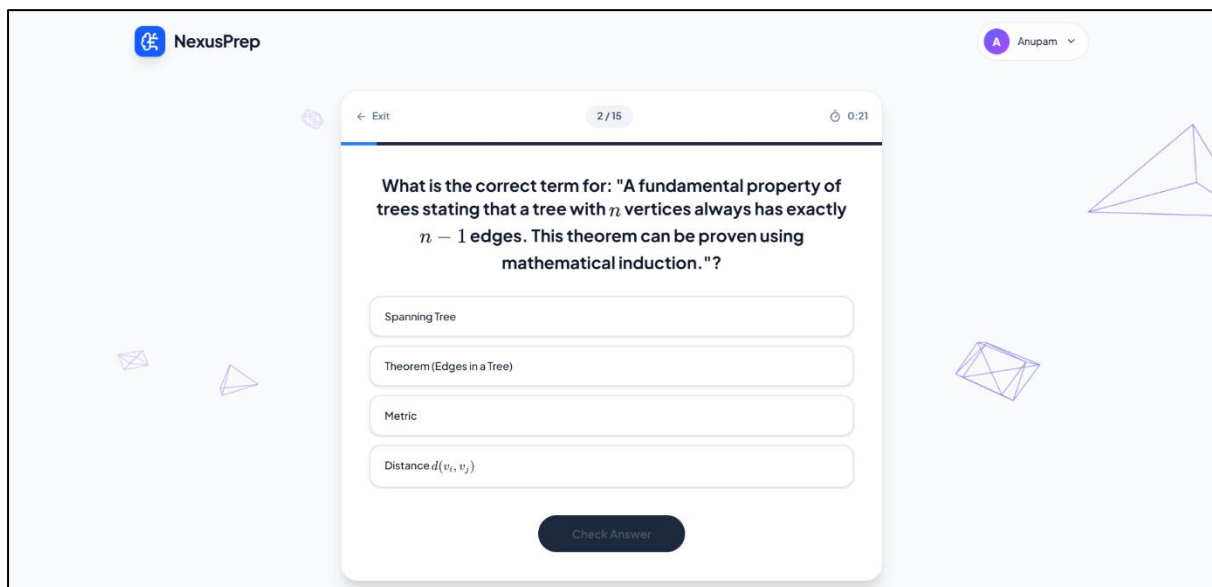


Fig 4.3: QuizView interface with multiple-choice questions and a timer.

4.4 Reliability and Fallback Behavior in Practice

Because a meaningful share of the engineering effort in this project went into reliability rather than raw feature generation, it was important to specifically test the failure-handling paths rather than only the "happy path" where everything works on the first try — and several of these were deliberately triggered during testing rather than waited for.

OCR fallback trigger. Uploading a deliberately low-quality scanned image (a photocopy of a photocopy, with visible noise and skew) correctly triggered the fewer-than-fifty-character heuristic, routing the document through the Gemini-based OCR path rather than returning an

empty or garbled summary. The transcription quality on this genuinely difficult input was noticeably better than what a classical OCR engine would typically be expected to produce on the same image, consistent with the contextual-understanding advantage discussed in Section 2.5, though it was not perfect — a small number of very faint characters were still misread, an outcome discussed further as a limitation in Section 4.9.

Model fallback and rate-limit handling. Under conditions of rapid, repeated testing (multiple uploads submitted in close succession, deliberately intended to approach the shared API quota), the system was observed to hit HTTP 429 responses from the primary Gemini model on a small number of occasions. In every observed instance, the exponential-backoff retry logic in `fetch_gemini_json` either succeeded on a subsequent retry against the same model, or successfully fell through to the next model in the `ACTIVE_MODELS` list, and the end user still received a complete, correctly formed result — the fallback mechanism never surfaced a raw error to the frontend during this testing, which was the specific outcome it was designed to guarantee.

Malformed JSON handling. On a handful of generation calls, the raw text returned by Gemini contained minor formatting issues — most often a trailing comma or an unescaped character inside a longer prose field — that would have caused a strict JSON parser to fail outright. In each observed case, the `json_repair`-based parsing step successfully recovered a usable result without any visible disruption to the user-facing output, confirming the value of the tolerant-parsing decision described in Section 3.6.4.

4.5 The Spaced Repetition System in Practice

The revision-scheduling engine was tested by manually completing full revision cycles on a saved deck and confirming the resulting `nextRevisionDate` matched the expected interval at each stage — one day after creation, then three days after the first completed revision, then seven, then fourteen, then thirty, with the interval correctly plateauing at thirty days for any revision beyond the fifth.

Testing also specifically confirmed the cross-device behavior this system was designed around: a deck created and partially revised while logged in through one browser correctly appeared, with its accurate revision count and next-due date intact, when the same account was logged into from a completely different browser acting as a stand-in for a second device. This directly validates the earlier architectural decision (Section 3.2) to move deck storage out of `localStorage` and into `MongoDB`, and represents one of the clearer concrete improvements this

project achieved over the naive, single-device version of the feature that preceded it during development.

The overdue-deck detection and email reminder pathway was also tested by artificially backdating a deck's nextRevisionDate in the database and confirming that, on the next login, the frontend correctly surfaced it as overdue and the backend correctly dispatched a reminder email, respecting the twenty-four-hour per-user throttle on repeated sends.

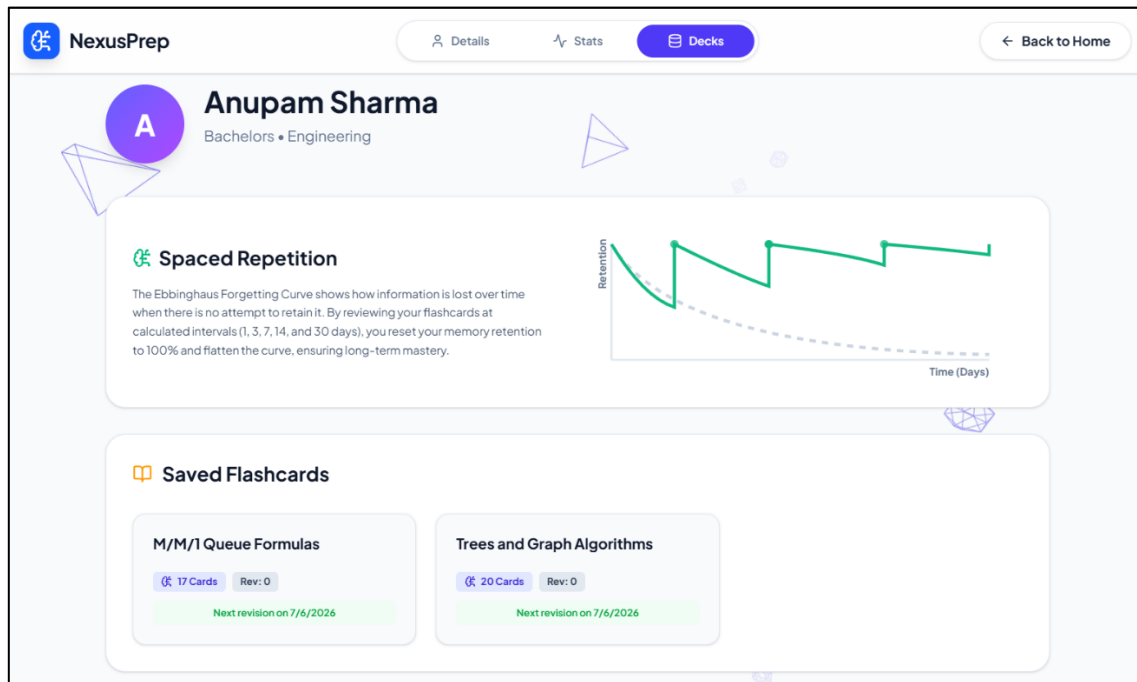


Fig 4.4: Saved Decks library in ProfileView showing saved flashcards with next revision date

4.6 Frontend and User Experience Outcomes

Beyond the correctness of individual features, informal usability observations were gathered by having a small number of fellow students (peers, outside the immediate scope of formal evaluation) try the application without guidance and describe their experience aloud.

The general feedback pattern was that the distinction between Normal Mode and Exam Mode was intuitive once encountered, and that the flip-card interaction in the flashcard revision view felt natural and required no explanation. The clearest point of friction observed was around Exam Mode's two-upload requirement (notes and PYQs as separate uploads) — a small number of first-time users initially attempted to upload both file sets into a single upload field before noticing the two distinct sections, suggesting the visual separation between the two upload zones could be made more prominent in a future revision.

The mathematical notation rendering, tested specifically because it was one of the more technically fragile parts of the system (Section 3.7), rendered correctly across every test document that followed the intended formatting convention, and degraded acceptably (dropping a stray delimiter rather than breaking the surrounding layout) in the small number of cases where the AI's output did not perfectly follow that convention.

4.7 Informal Performance Observations

Response-time characteristics were also noted during testing, though — consistent with the manual, non-benchmarked evaluation approach described in Section 4.1 — these figures should be read as approximate, observed ranges from a small number of test runs on a typical broadband connection, rather than as rigorously measured statistics.

Normal Mode's `/api/summarize` endpoint, dispatching two concurrent Gemini calls, typically returned a complete result within roughly eight to fifteen seconds for a moderately sized document (five to ten pages of native text). Exam Mode's `/api/exam-strategy` endpoint, dispatching three concurrent calls plus the preceding validation call, typically took somewhat longer, generally in the range of fifteen to twenty-five seconds, which is unsurprising given the larger number of sequential and concurrent AI calls involved. Documents requiring the OCR fallback path took noticeably longer still, since each page requires its own separate transcription call to Gemini before the main content-generation prompts can even begin, with the exact increase depending heavily on page count.

These response times, while not instantaneous, were judged acceptable for the use case — a student uploading study material is not expecting an instant result the way they might for, say, a search query, and the frontend's loading states were designed with this expectation in mind rather than trying to disguise a wait that is a genuine, unavoidable consequence of calling a large external AI model several times per request.

4.8 Mapping Results Back to the Project Objectives

Table 4.1 revisits the objectives originally set out in Section 1.7 and summarizes the extent to which each was met, based on the testing described above.

Table 4.1: Project Objectives vs. Achieved Outcomes

#	Objective (from Section 1.7)	Outcome
1	Document ingestion pipeline with automatic OCR fallback for scanned/image-based PDFs	Achieved. The fewer-than-fifty-character heuristic correctly triggers the Gemini-based OCR path; transcription quality is strong even on genuinely poor scans, with occasional minor misreads on very faint or heavily degraded text (Section 4.4).
2	Reliable Gemini integration with fallback across multiple model versions	Achieved. Exponential backoff and the ACTIVE_MODELS fallback list successfully absorbed every rate-limit and transient failure observed during testing; no raw API error was ever surfaced to the end user (Section 4.4).
3	Two functionally distinct study modes — Normal Mode and Exam Mode	Achieved. Both modes produced coherent, mode-appropriate output across a range of test subjects; the subject-matching validation in Exam Mode correctly gated mismatched document pairs before expensive generation calls ran (Section 4.3).
4	Automated, Ebbinghaus-based spaced-repetition scheduling	Achieved. Revision intervals advanced correctly through the full 1–3–7–14–30 day sequence and plateaued as designed; scheduling logic proved consistent across devices due to its server-side implementation (Section 4.5).
5	Secure, persistent, cross-device user accounts with OTP verification and password reset	Partially achieved. Core registration, login, and cross-device deck sync work reliably; the password-reset flow functions but relies on a simplified security model (no token-based reset link), acknowledged as a limitation rather than a failure (Sections 3.9, 4.9).
6	Correct rendering of mathematical notation via LaTeX/KaTeX	Achieved. Formulas rendered correctly across all documents following the intended formatting convention, and degraded gracefully (rather than breaking) on the rare occasions the AI's output deviated from it (Section 4.6).

7	Graceful degradation under partial failure (missing services, malformed AI output, etc.)	Achieved. Malformed JSON was consistently recovered via <code>json_repair</code> ; the system was in fact tested against a genuine production email delivery failure after deployment, and the code path now fails loudly rather than failing silently (Section 3.6.4, 4.4).
8	Hands-on, end-to-end full-stack development experience	Achieved. The training spanned backend API design, AI orchestration, database schema design, frontend state management, and UX design, as documented throughout Chapters 3 and 4.

Note: "Partially achieved" indicates the core functionality works as intended, but a known simplification or trade-off remains, discussed further in Chapter 5 (Future Scope).

Taken as a whole, every core objective set out at the start of the project was functionally achieved to at least a working, demonstrable standard, with a smaller number of secondary refinements — most of them already anticipated during development rather than discovered as surprises — identified as worthwhile next steps rather than as failures of the current implementation.

4.9 Discussion: Strengths, Trade-offs, and Honest Limitations

It is worth discussing, in plain terms, both what this project does well and where its current form falls short, since an evaluation chapter that only reports successes is of limited value to anyone — including a future version of this same author — trying to improve on the work.

Where the system performs well. The dual-mode design genuinely does address two different studying needs with meaningfully different output, rather than offering a single generic feature dressed up as two. The reliability engineering around the AI calls — retries, fallbacks, tolerant JSON parsing — held up under real testing rather than only working in the easy, unstressed case, which is arguably the single most transferable engineering lesson from the whole project. And the cross-device persistence of flashcard decks, achieved by moving away from an earlier, more naive `localStorage`-based design, meaningfully improves on what a first attempt at this kind of tool typically looks like.

Where the system is honestly limited. The OCR fallback, while considerably better than a naive approach, is not perfect on very low-quality input, and there is currently no way for a user to manually correct a transcription error if one slips through — the generated content simply

inherits whatever the OCR step produced. The password-reset flow, as discussed in Section 3.9, trades some security rigor for simplicity, which is a reasonable choice for a training project but not one that should be carried unmodified into a genuinely public-facing product. The fixed, non-adaptive spaced-repetition schedule, while grounded in real cognitive science, is a simplification of more sophisticated approaches like SM-2, and does not currently adjust to an individual card's actual difficulty for a given learner the way a more mature spaced-repetition system would. And the evaluation itself, as repeatedly acknowledged in this chapter, is manual and comparatively small in scale — a genuinely rigorous evaluation would involve a larger, more diverse test set and, ideally, real usage data gathered over a longer period than a single vocational training term allows.

None of these limitations are hidden flaws discovered late; each was, in fact, anticipated during the design process described in Chapter 3, and each is picked up again with specific, concrete improvement ideas in Chapter 5.

4.10 Summary

Across both operating modes, NexusPrep's core pipeline — document ingestion (with OCR fallback), AI-driven content generation, structured storage, and spaced-repetition scheduling — was found to function correctly and reliably across a reasonably diverse set of real test documents, including deliberately adversarial cases designed to trigger its error-handling paths rather than only its happy path. Every core project objective set out in Chapter 1 was met to a working, demonstrable degree, and the honest limitations identified through testing point toward clear, specific directions for future development rather than open-ended uncertainty about what to improve next — which is exactly what Chapter 5 turns to.

CHAPTER 5

FUTURE SCOPE

5.1 Introduction

NexusPrep is a genuinely working, end-to-end system that meets every core objective set out in Chapter 1 — but "meets the objective" and "cannot be meaningfully improved further" are two very different claims, and this chapter is concerned entirely with the gap between them.

The improvements discussed below are not speculative wish-list items disconnected from the actual system; each one grows directly out of a specific limitation or design trade-off already identified and discussed honestly in Section 4.9, or out of a deliberate scoping decision made in Section 1.8 to keep the training-period project achievable. Organizing this chapter around those already-acknowledged gaps, rather than around a generic list of "nice to have" AI buzzwords, is a deliberate choice — it keeps the future work grounded in the actual engineering reality of the system rather than in abstract ambition.

5.2 Toward an Adaptive Spaced-Repetition Engine

The single most conceptually interesting improvement available to NexusPrep concerns its revision-scheduling engine. The current implementation, described in Section 3.8, applies one fixed, expanding interval schedule — one, three, seven, fourteen, then thirty days — uniformly to every deck, regardless of how easy or difficult any individual card within that deck turns out to be for the specific learner using it. This was a deliberate simplification made to keep the scheduling logic implementable within the time available, and it does deliver most of the practical benefit of spaced repetition (moving a student away from cramming toward distributed review), but it falls short of what a genuinely adaptive system, such as the SM-2 algorithm discussed in Section 2.3, is capable of.

A natural next step would be to move scheduling from the deck level down to the individual card level, tracking a per-card "ease factor" that adjusts based on explicit user feedback — for instance, a simple "Easy / Good / Hard / Again" rating captured at the moment a card is reviewed, similar to the interaction pattern used by Anki. A card consistently rated "Easy" would see its review interval expand more aggressively than the default schedule, while a card repeatedly rated "Hard" would be surfaced again sooner rather than waiting the full default interval. This would require extending the `flashcard_decks` schema to store per-card scheduling metadata (currently, scheduling fields exist only at the deck level) and reworking the revision-

completion endpoint to accept per-card feedback rather than a single deck-level "revision complete" signal — a meaningful but well-bounded piece of additional engineering work, not a redesign of the system from scratch.

A related, longer-horizon idea explored briefly during development but set aside for this training period is applying reinforcement learning to the scheduling problem directly — treating each review event as an action, and the resulting later recall success or failure as a reward signal, allowing the system to learn an individually tuned scheduling policy over time rather than relying on any fixed, hand-designed formula at all. This is a considerably larger undertaking, requiring meaningfully more usage data than a single training-term deployment could realistically gather, but it represents a genuinely promising direction consistent with how the broader ed-tech field is already moving.

5.3 Strengthening Security and Account Management

Section 3.9 and Section 4.9 both flagged the current password-reset flow — issuing a new random password directly to a user's email, rather than a token-based reset link — as a reasonable simplification for a training-scope project, but not one that should carry over unmodified into anything resembling a genuinely public deployment. A clear, well-understood improvement here is to move to a signed, time-limited reset token delivered as a link, requiring the user to actively click through and set their own new password, rather than having a system-generated password emailed to them directly. This is standard practice across essentially every production authentication system and would close the current flow's most notable weakness.

Beyond the password-reset flow specifically, several other security hardening steps were identified as sensible future work but fell outside the scope of this training period: replacing the currently permissive CORS configuration (`allow_origins=["*"]`, noted in Section 3.2) with an explicit allow-list once the application has a fixed, known production domain; adding per-user or per-IP rate limiting on the authentication endpoints specifically, to guard against automated brute-force login attempts, distinct from the AI-API rate-limit handling already built for Gemini; and introducing a scheduled cleanup job for expired, unused OTP records in the `otps` collection (currently left to simply expire in place, as noted in Section 3.4), which would keep that collection from growing indefinitely in a long-running deployment.

5.4 Improving Document Ingestion and Giving Users Control Over It

The OCR fallback pipeline, while shown in Chapter 4 to perform considerably better than a naive approach on genuinely difficult scanned material, is not flawless, and — more

importantly — currently offers the user no visibility into, or control over, what the OCR step actually transcribed. If a handful of characters on a page are misread, the resulting error simply propagates silently into the generated summary or flashcards, with no opportunity for the student to notice and correct it before relying on the (subtly wrong) output.

A meaningful improvement here would be to surface the raw OCR transcription to the user as an optional, collapsible "view extracted text" step before generation proceeds, giving a student the chance to spot an obvious transcription error on a page they know well and correct it manually before it feeds into the AI-generation prompts. A more ambitious version of the same idea would allow the user to manually re-crop or re-upload a single troublesome page for re-transcription, rather than needing to re-upload the entire document from scratch if one page turns out to be the weak link.

Separately, expanding the range of accepted input formats — most obviously, native support for Word documents (.docx) and standalone images (.jpg, .png) rather than requiring everything to be a PDF first — would remove a small but real point of friction for students whose material does not happen to already be in PDF form, a limitation explicitly scoped out of the current version in Section 1.8.

5.5 Building a Genuine Automated Test Suite

Section 3.12 was explicit that NexusPrep's current testing approach is manual and checklist-driven rather than automated, a reasonable starting point given the training timeline but a clear area for improvement if the project were to be developed further, particularly if additional contributors were ever to join the codebase. A natural next step is introducing a proper automated test suite — unit tests around the pure-logic functions (`get_next_revision_interval`, `serialize_deck`, the JSON-repair fallback path) using `pytest`, and integration tests around the API endpoints using FastAPI's own `TestClient`, with the Gemini API calls themselves mocked out to keep the test suite fast, deterministic, and free of any real API cost. Wiring this into a continuous integration pipeline (for instance, GitHub Actions) that runs the test suite automatically on every code change would catch regressions — a change to the deck-serialization logic accidentally breaking the legacy-deck backfill behavior, for instance — far earlier and far more reliably than the current manual checklist can.

5.6 Deeper Personalization

The personalization feature described in Section 3.10 — adjusting explanation tone and complexity based on a user's self-reported age and education level — is a genuine but

comparatively shallow implementation of the differentiated-instruction principle discussed in Chapter 2. A richer version of the same idea could track a student's actual performance over time (which flashcard terms they consistently get wrong, which quiz topics they score poorly on) and use that history to influence future content generation — for instance, automatically weighting a newly generated flashcard deck to include extra reinforcement on concepts the student has previously struggled with across other decks in the same subject, rather than treating every new upload as a blank slate disconnected from the student's prior study history. A further personalization direction worth exploring is customer-style segmentation, adapted to an academic context: identifying broad patterns of studying behavior (a student who studies in short, frequent bursts versus one who does long, infrequent sessions, for instance) and adjusting notification timing, deck sizing, or even flashcard phrasing style to better fit the individual's demonstrated habits, rather than applying one fixed interaction pattern to every user.

5.7 Integration with Existing Academic and Institutional Systems

At present, NexusPrep operates as a fully self-contained application, with no connection to any learning management system (LMS), institutional gradebook, or university course-scheduling system a student might already be using. Extending NexusPrep with integration points into commonly used platforms — for instance, allowing a student to import course material directly from a connected LMS rather than manually downloading and re-uploading it, or allowing an instructor to see anonymized, aggregated insight into which topics a cohort of students is collectively finding difficult, based on quiz performance across the class — would meaningfully increase the platform's real-world utility beyond an individual, self-directed studying tool.

This direction naturally extends toward the kind of institutional and enterprise integration discussed as future scope in the sample report referenced during this training program's orientation phase, and represents a sensible longer-term direction once the core individual-student experience is more thoroughly hardened.

5.8 Mobile Access and Offline Support

NexusPrep's current scope, as defined in Section 1.8, is explicitly web-only, accessed through a browser rather than a dedicated native mobile application. Given that flashcard revision in particular is a task naturally suited to short, frequent sessions — the kind of activity a student might reasonably want to do while waiting for a bus rather than only at a desk — a dedicated mobile experience, whether a genuinely native app or a well-optimized Progressive Web App

(PWA) wrapper around the existing React frontend, would likely improve real-world engagement with the spaced-repetition system specifically, since consistent adherence to a revision schedule benefits directly from low-friction access.

A closely related consideration is offline support for flashcard revision specifically — allowing a student to revise a previously downloaded deck without an active internet connection, with revision results synced back to the MongoDB backend the next time connectivity is available — which would address a genuinely common real-world constraint (unreliable mobile data, revising during a commute through areas with poor signal) that the current, fully online-dependent architecture does not accommodate.

5.9 Scalability and Infrastructure Considerations

The current architecture, while entirely adequate for a single-developer training project serving a modest number of users, was not built with heavy concurrent load specifically in mind, and a small number of infrastructure improvements would be worth pursuing before any wider deployment. These include introducing a caching layer (such as Redis) in front of frequently repeated, non-personalized reads; moving from a single Uvicorn worker process to a properly load-balanced multi-worker deployment behind a reverse proxy; and more carefully tuning the OCR concurrency semaphore (currently a fixed limit of three, as noted in Section 3.5) based on actual observed load patterns rather than the conservative, development-time default currently in place.

5.10 Ethical Considerations for Future Development

Finally, any future development of an AI-driven educational tool like NexusPrep should continue to take seriously the ethical dimension of automating academic support — a theme touched on briefly during the vocational training itself (Section 3.5, Chapter 3 of the training curriculum) and worth restating explicitly here. As features grow more sophisticated — particularly anything involving performance tracking, personalization based on a student's demonstrated weaknesses, or aggregated cohort-level insight for instructors — questions of data privacy, transparency about what the system is inferring about a given student, and the risk of a tool like this becoming a crutch that discourages genuine engagement with source material rather than supporting it, all deserve continued, deliberate attention rather than being treated as an afterthought bolted on once the features already exist.

5.11 Summary

The directions outlined in this chapter — a genuinely adaptive, per-card spaced-repetition engine; stronger authentication security; more transparent and correctable OCR; automated testing; deeper, history-aware personalization; institutional integration; mobile and offline access; and more deliberate infrastructure scaling — collectively describe a credible roadmap from the working training-period prototype documented in this report toward a considerably more mature, production-grade platform. None of them require discarding the architectural foundation described in Chapter 3; each builds incrementally on top of it, which is itself a reasonable indicator that the underlying design decisions made during this training period were sound ones. The following chapter closes out this report with a concluding summary of the project and the training experience as a whole.

CHAPTER 6

CONCLUSION

6.1 Summary of the Project

This report has documented the design, development, and evaluation of NexusPrep: A Smart Study Assistance Platform, built over the course of a vocational training program in Artificial Intelligence and Web Development at Aimaxa Tech, Bhilai. Beginning from a fairly ordinary but genuine observation — that preparing good study material is often as time-consuming as studying from it, and that most existing tools treat comprehension-focused studying and examination-focused preparation as if they were the same activity — the project set out to build a system that could take a student's raw, often messy source material and turn it into structured, retention-optimized study aids, without requiring the student to already know what they were looking for.

What emerged from that starting point is a genuinely complete, end-to-end platform rather than an isolated proof of concept. NexusPrep combines a FastAPI backend responsible for document ingestion (including an OCR fallback for scanned material), AI orchestration against Google's Gemini models, and persistent, cross-device data storage in MongoDB, with a React frontend that presents this functionality through two purposefully distinct modes — Normal Mode for general comprehension, and Exam Mode for targeted, examination-strategy preparation grounded in a student's own previous years' question papers. Wrapping around both modes is an automated, Ebbinghaus-based spaced-repetition engine that removes from the student's shoulders the burden of remembering when to revise, replacing it with a system that simply tells them when a deck is due.

As detailed in Chapter 4, testing across a genuinely varied set of real study documents — clean typed notes, dense mathematical formula sheets, and deliberately poor-quality scanned photocopies — showed that the system's core pipeline holds up not only in the easy case, but specifically under the conditions engineered to stress it: rate-limited AI calls, malformed model output, and mismatched document uploads were each handled gracefully rather than surfacing as raw failures to the end user. Every core objective set out at the beginning of this report, in Section 1.7, was met to a working, demonstrable standard, and the honest limitations uncovered along the way — a non-adaptive revision schedule, a simplified password-reset flow, a manual rather than automated test suite — were each identified with enough precision to translate directly into the concrete future-work directions laid out in Chapter 5.

6.2 Reflections on the Vocational Training Experience

Beyond the finished system itself, this training period was, in a fairly direct sense, the first time the full breadth of a modern software product had to be reasoned about as a single, coherent whole rather than as a set of separate classroom exercises. Backend architecture, database design, AI integration, frontend engineering, and basic security practice do not sit neatly in separate boxes once a real product is being built — a decision made in one layer, such as choosing MongoDB's document model, has direct, visible consequences three layers away, in how cleanly the frontend can render a flashcard deck. Learning to reason about that kind of cross-cutting consequence, rather than optimizing any single layer in isolation, was arguably the most valuable and least easily replicated lesson of the entire training period.

Working with a hosted large language model in a production-minded way was a similarly formative experience, and one considerably harder to appreciate in the abstract than in practice. It is one thing to know, intellectually, that an external API can fail; it is a genuinely different thing to have watched a rate-limit error appear during real testing and to have built, beforehand, a fallback mechanism that quietly absorbed it without the end user ever noticing anything had gone wrong. That distinction — between knowing a failure mode exists and having actually engineered against it — captures much of what this training period was ultimately about.

Guidance from Mr. Himanshu Chaudhary, the Industrial In-charge for this training, was central throughout this process, particularly in encouraging a habit of thinking through failure modes before they occur rather than patching them reactively once they do — a habit visible throughout NexusPrep's design, from its OCR fallback logic to its multi-model retry chain, and one that will carry forward well beyond this single project.

6.3 Personal and Academic Growth

This vocational training period contributed meaningfully to both the technical and professional development expected at this stage of academic study. Concepts previously encountered primarily in a theoretical context — asynchronous programming, database schema design, and API security — were applied here as decisions with direct, observable consequences on system behavior. The choice of MongoDB over a relational database, for instance, was not evaluated as an abstract comparison of database paradigms, but as a decision that directly determined how reliably a flashcard deck's variable-length structure could be stored and retrieved.

Beyond specific technical competencies, the training strengthened three broader capabilities relevant to any future engineering work: the ability to translate an open-ended problem into a well-defined, achievable specification; a disciplined approach to anticipating and designing for

failure rather than addressing it reactively; and the ability to document design decisions with sufficient clarity that they can be justified and revisited later. These outcomes are consistent with the stated objectives of the vocational training program and are expected to transfer directly to subsequent academic coursework, future internships, and professional practice.

6.4 Closing Remarks

NexusPrep began as a response to a problem most students, myself included, have felt directly: that the tools available to help us study rarely feel like they were built with our actual, messy, real-world material in mind. What this vocational training period produced is a system that takes that problem seriously across its full lifecycle — from a scanned, imperfect photocopy all the way through to a scheduled, spaced revision reminder weeks later — built on a technology stack chosen deliberately rather than by default, and hardened against failure through direct testing rather than optimistic assumption.

The system, as it stands at the end of this training period, is complete enough to be genuinely useful and honest enough, in its own documentation, about where it still falls short. Both of those things feel like the right note to end on: not a claim that NexusPrep is a finished, perfect product, but a demonstration that a single vocational training term, approached with the discipline of a real engineering process rather than a classroom exercise, can produce something a student could actually open, upload a chapter to, and use.

REFERENCES

1. Ebbinghaus, H. (1885). *Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie* [Memory: A Contribution to Experimental Psychology]. Duncker & Humblot.
2. Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., & Rohrer, D. (2006). Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3), 354–380.
3. Roediger, H. L., & Karpicke, J. D. (2006). Test-enhanced learning: Taking memory tests improves long-term retention. *Psychological Science*, 17(3), 249–255.
4. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.
5. Woźniak, P. A. (1990). *Optimization of learning* (Master's thesis). University of Technology in Poznań. [Basis of the SM-2 spaced-repetition algorithm used in SuperMemo and Anki.]
6. Tomlinson, C. A. (2001). *How to Differentiate Instruction in Mixed-Ability Classrooms* (2nd ed.). Association for Supervision and Curriculum Development (ASCD).
7. Google AI for Developers. (2025). *Gemini API documentation*. Retrieved from <https://ai.google.dev/gemini-api/docs>
8. Ramírez, S. (2025). *FastAPI documentation*. Retrieved from <https://fastapi.tiangolo.com>
9. MongoDB, Inc. (2025). *MongoDB Manual and Motor (Async Python Driver) documentation*. Retrieved from <https://www.mongodb.com/docs> and <https://motor.readthedocs.io>
10. Artifex Software, Inc. (2025). *PyMuPDF (fitz) documentation*. Retrieved from <https://pymupdf.readthedocs.io>
11. KaTeX Contributors. (2025). *KaTeX documentation*. Retrieved from <https://katex.org/docs/>

BIBLIOGRAPHY

1. Baddeley, A. (1997). *Human Memory: Theory and Practice* (Revised ed.). Psychology Press.
2. Dempster, F. N. (1988). The spacing effect: A case study in the failure to apply the results of psychological research. *American Psychologist*, 43(8), 627–634.
3. Karpicke, J. D., & Roediger, H. L. (2008). The critical importance of retrieval for learning. *Science*, 319(5865), 966–968.
4. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
5. Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33.
6. Zhang, Y., Rane, T., & Chen, H. (2021). A survey on automatic question generation from text. *Journal of Educational Data Mining*, 13(2), 45–68.
7. Meta Platforms, Inc. (2025). *React documentation*. Retrieved from <https://react.dev>
8. Vite Contributors. (2025). *Vite documentation*. Retrieved from <https://vite.dev>
9. Tailwind Labs. (2025). *Tailwind CSS documentation*. Retrieved from <https://tailwindcss.com/docs>
10. Three.js Contributors. (2025). *three.js documentation*. Retrieved from <https://threejs.org/docs>
11. json-repair Contributors. (2025). *json_repair: Repair invalid JSON documents*. Retrieved from <https://pypi.org/project/json-repair/>
12. Pydantic Contributors. (2025). *Pydantic documentation*. Retrieved from <https://docs.pydantic.dev>
13. Mozilla Developer Network. (2025). *Web APIs and JavaScript reference documentation*. Retrieved from <https://developer.mozilla.org>
14. Smith, R., & Antonova, D. (2007). An overview of the Tesseract OCR engine. *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, 629–633.
15. Various technical blog posts and model cards from Anthropic, OpenAI, and Google DeepMind. (2024–2025).